JÖNKÖPING UNIVERSITY
School of Engineering

Exploring the performance gap: How animation implementation affects the CPU and RAM usage in mobile applications

Among cross-platform and native development approaches

Main subject area: *Computer Engineering*
Author: *Sabina Ametova, Tim Lindström*
Supervisor: *Garrit Schaap*
JÖNKÖPING 2023 Feb

This final thesis has been carried out at the School of Engineering at Jönköping University within Computer Engineering. The authors are responsible for the presented opinions, conclusions and results.

Examiner:	Johannes Schmidt
Supervisor:	Garrit Schaap
Scope:	15 hp
Date:	2023-05-31

Abstract

Mobile applications are a critical part of everyday life. Animations in mobile applications demand optimized performance, which developers are persistently striving to achieve. This study examines the performance of mobile applications developed using native approaches for iOS and Android, as well as the Flutter framework for cross-platform approach, while executing animations. This was motivated by the fact that while cross-platform development offers advantages in terms of development efficiency, it often results in decreased performance. Since the field of mobile development is continuously evolving, it is important to explore new capabilities of mobile technologies.

The purpose of this study is to understand the differences in CPU and RAM usage between native and cross-platform applications by examining two types of animation implementations, thereby aiming to identify the causes of the performance gap and propose solutions to decrease the performance gap. This study involved experiments, a literature study and an analysis of the results to address the research questions and fulfill the purpose.

The findings of this study showed that the cross-platform approach resulted in higher CPU and RAM usage than the native approaches. However, the performance gap was less noticeable in one of the animation implementations. Thus indicating that the performance of mobile applications can be significantly influenced by the choice of animation implementation. Therefore, a strategic approach to mitigating the performance gap could involve careful selection of Flutter widgets or the incorporation of efficient libraries.

Keywords: animations, app development, cross-platform, Flutter, mobile development, performance

Acknowledgements

We would like to start this report with showing gratitude and appreciation to all who were involved in and supported us during this thesis. Firstly, a big thank you to Garrit Schaap for providing guidance and expertise within this area of research. We would also like to thank Hanna Hansson and Isac Sélea at Etteplan for giving us additional guidance and supervision in the thesis process. Lastly, we would like to give a huge thank you to our close friend Rita, without you none of this would have been possible.

Table of Content

ABSTRACT ..... **II**

ACKNOWLEDGEMENTS ..... **III**

LIST OF ABBREVIATIONS ..... **VI**

LIST OF FIGURES ..... **VII**

LIST OF TABLES ..... **VIII**

1 INTRODUCTION **1**

1.1 RELATED RESEARCH 2
1.2 PROBLEM STATEMENT 4
1.3 PURPOSE AND RESEARCH QUESTIONS 5
1.4 SCOPE AND LIMITATIONS 6
1.5 DISPOSITION 7

2 METHOD AND IMPLEMENTATION **8**

2.1 DESIGNING THE ANIMATION 8
2.2 ARTIFACT DEVELOPMENT 10
2.2.1 Native Animation Implementation *12*
2.2.2 Lottie Animation Implementation *14*
2.3 DATA COLLECTION 15
2.3.1 The Experiments *15*
2.3.2 Literature Study *18*
2.3.3 Technologies *19*
2.3.4 Metrics and Measuring Tools *20*
2.3.5 Devices *21*
2.4 DATA ANALYSIS 21
2.5 VALIDITY AND RELIABILITY 22
2.6 CONSIDERATIONS 23

3 THEORETICAL FRAMEWORK **25**

3.1 NATIVE DEVELOPMENT 25
3.1.1 Android *25*
3.1.2 iOS *26*
3.2 CROSS-PLATFORM DEVELOPMENT 27
3.3 FLUTTER 29
3.4 LOTTIE ANIMATION 30
3.5 ADOBE AFTER EFFECTS 30

4 RESULTS **31**

4.1 NATIVE ANIMATION ANALYSIS 31
4.2 LOTTIE ANIMATION ANALYSIS 32

5 DISCUSSION **33**

5.1 RESULT AND ANALYSIS DISCUSSION 33

5.1.1 Research Question 1 ..... *33*
5.1.2 Research Question 2 ..... *36*
5.1.3 Research Question 3 ..... *38*
5.2 METHOD DISCUSSION 40
6 CONCLUSIONS AND FURTHER RESEARCH 43
6.1 CONCLUSIONS 43
6.1.1 Practical Implications *43*
6.1.2 Scientific Implication *44*
6.2 FURTHER RESEARCH 44
7 REFERENCES 45
8 APPENDIX 50

List of Abbreviations

Abbreviation	Meaning	Page
OS	operating system	1
CPU	central processing unit	1
RAM	random access memory	1
FPS	frames per second	1
UI	user interface	4
MVP	minimal viable product	6
XML	extensible markup language	12
IDE	integrated development environments	18
MB	megabytes	20
SDK	software development kit	25
PWA	progressive web application	27
DSL	domain specific language	28
GPM	general-purpose modeling	28
UML	unified modeling language	28
JIT	just-in-time	29
AOT	ahead-of-time	29
ARC	automatic reference counting	38
CC	concurrent copying	38
ART	android runtime	38

List of Figures

Figure	Description	Page
1	The flow of this study's animation from start to finish.	10
2	Screenshots displaying the layout of the start screen, Lottie Animation screen and the Native Animation screen within the mobile applications.	11
3	Flowchart illustrating the profiling process of each mobile application.	17

List of Tables

Table	Description	Page
1	Mobile applications by operating system (row) and development approach (column).	10
2	Animation performance datasets by mobile application (row) and animation implementation (column).	15
3	An overview of the technologies used.	20
4	Tools for measuring the RAM and CPU usage for the Android and iOS platforms.	21
5	Display of the different mobile devices that were used in the study.	21
6	A description of the five cross-platform development approaches and the most popular frameworks using the approaches.	27
7	CPU usage results from Native Animation performance experiment for each mobile device.	31
8	RAM usage results from Native Animation performance experiment for each mobile device.	31
9	CPU usage results from Lottie Animation performance experiment for each mobile device.	32
10	RAM usage results from Lottie Animation performance experiment for each mobile device.	32

1 Introduction

Mobile devices and their associated applications have become an integral part in the daily lives of the vast majority of individuals. The mobile devices used today are largely split between two major mobile *operating systems* (OS); Android and Apple iOS (Menegassi & Endo, 2020). To feature an application on both of these OS and their respective platforms requires a mobile application to be developed several times, in different programming languages, resulting in multiple different code bases that needs to be maintained and updated separately (Huber et al., 2020). A solution to this issue is the use of cross-platform frameworks for developing the applications (Ciman et al., 2014). By using a cross-platform framework the developed code can be compiled to multiple platforms from a single codebase (Shah et al., 2019), lowering the development time and the cost of production (Ciman et al., 2014).

While the use of cross-platform frameworks offers advantages in terms of development efficiency, it is important to consider the performance impact. A study of cross-platform development conducted by Xanthopoulos and Xinogalos (2013, p. 213) stated that ”The ultimate goal of cross-platform mobile application development is to achieve native application performance and run on as many platforms as possible”. However, this goal has not been fully achieved, as of today the use of cross-platform frameworks for developing mobile applications usually results in decreased performance compared to a native development approach (Barros et al., 2020; Biørn-Hansen et al., 2019a; Huber et al., 2020).

The term *performance*, when referring to a mobile application, includes metrics such as *Central Processing Unit* (CPU) and *Random Access Memory* (RAM) usage, battery consumption, *Frames Per Second* (FPS) and response time. In this study, the performance will exclusively refer to the metrics of CPU and RAM usage. The CPU usage indicates how many processes that are currently running on the device and how high of a percentage of the CPU every process is taking. An application with high CPU usage can affect other processes running on the device, leading to a worsend performance (Willocx et al., 2015). The RAM usage reveals how much memory the mobile application allocates while running (Willocx et al., 2015). If an application uses too much RAM memory, the usability of the application can be affected, and as a result forcing the application to slow down (Ahti et al., 2016). These metrics are thereby not only relevant for this study but also hold broader significance for mobile development, as they provide insights into the resource utilization of mobile applications and their impact on the device.

How big the loss of performance is between cross-platform and native applications depends on the features used (Biørn-Hansen et al., 2019b). It is thereby relevant to study each feature separately and extensively. One of these features are animations, which are an integral part of modern mobile applications for enhancing the user experience (Biørn-Hansen et al., 2019a). Animations can vary significantly in complexity, from subtle effects that provide feedback or show progress, to complex scenes that might require a dedicated animation engine. In this study the focus is on a level of complexity that is high enough to noticeably affect the performance of a mobile application, but not so high that it necessitates the use of external animation engines. The aim of the animation is to create a significant workload for the purposes of the performance measurements while reflecting the kind of animations often found in real-world applications.

An analysis of the current state of the cross-platform mobile development approaches reveals that there exist many studies comparing native and cross-platform applications but only a few focus on animations, where the most recent and relevant ones are Andersson (2022), Biørn-Hansen et al. (2019a) and Huber et al. (2020). Earlier studies are continuously decreasing in relevance due to the rapid advancement of mobile devices as well as cross-platform solutions. Assessing new framework releases is crucial to bridge the existing knowledge gap and evaluate the extent to which cross-platform mobile applications can match the performance of native mobile applications. While there are many cross-platform solutions available, this study focuses exclusively on Flutter. The reason being that Flutter has become one of the most popular cross-platform framework as of April 2023 according to the number of stars in their official GitHub repository (152k), followed by React Native (109k) and Ionic (49k) (Github, n.d.-a; Github, n.d.-c; Github, n.d.-f). An additional reason for studying Flutter is the release of a new version in May 2022, Flutter 3, that has yet to be thoroughly studied (Flutter Team, 2022).

1.1 Related Research

Given the popularity of Flutter, it becomes essential to study its performance and efficiency. Particularly, the context of Flutter's use of animations is interesting given the findings from the work of Biørn-Hansen et al. (2019a), Andersson (2022), Nawrocki et al. (2021) and Huber et al. (2020).

The study of Biørn-Hansen et al. (2019a) compared the performance of three cross-platform mobile applications and two native applications through the use of a Lottie animation, navigation transitions and side menu animation. The cross-platform frameworks used were React Native, Ionic Framework and Xamarin Forms. The Lottie

animation used in their study is a complex vector animation created with motion graphics tools and stored in JSON format, allowing the animations to be used on multiple platforms (LottieFiles, n.d.-b). The result from their Lottie animation experiment found that all of the cross-platform mobile applications had higher memory usage compared to the native applications. Biørn-Hansen et al. (2019a) measured the CPU peak usage which differed significantly between the technologies used where only the Ionic Framework application had a higher CPU peak than the native Android application on the Android device. The Ionic Framework application was also the only application having a lower CPU peak than the native iOS application on the iOS device.

Biørn-Hansen et al. (2019a) did not include Flutter in their study of animations, but the work of Andersson (2022) and Nawrocki et al. (2021) reveals that Flutter also has a pattern of high processing and memory usage. In the study of Andersson (2022) a cross-platform application created with the Flutter framework is compared to a native Android application by comparing: (1) lists, (2) file decoding, (3) database storage, (4) database retrieval, (5) geo-location and (6) animations. The animation in the study consisted of re-scaling a star icon with native animation classes of Android and Flutter. The findings from the animation comparison showed that both the peak and mean CPU usage of the Flutter animation had a higher CPU usage, and the peak and mean RAM usage were multiple times higher than the animation in the native Android application.

The work of Huber et al. (2020) compared one native Android application with two cross-platform developed applications created with React Native and The Ionic Framework. The three mobile applications' performance was tested on three different Android devices with the use of three UI interactions: (1) opening or closing a navigation drawer, (2) screen transition, and (3) virtual scrolling of a list. The findings of their study showed that both the CPU usage and RAM usage was consistently higher for all UI interactions in the cross-platform mobile application compared to the native mobile application on all devices.

Nawrocki et al. (2021) compared three cross-platform mobile applications created with Flutter, React Native and Xamarin against two native mobile applications in iOS and Android. The performance comparison in the article tests the performance of the cross-platform frameworks by comparing different metrics of the mobile application: (1) build size, (2) launch time, (3) RAM usage, and (4) CPU load. The metrics were tested under a variety of conditions and workloads. The result of their study showed that the RAM usage of the Flutter mobile application was higher in all conducted tests compared to the native mobile applications. The CPU usage varied more where the Flutter mobile application was the only cross-platform application to beat the native Android implementation.

This study builds upon the methodology established by Biørn-Hansen et al. (2019a) in their examination of the Lottie animation. However not only is their approach replicated in this study, but extended by incorporating a performance analysis of animations implemented using native animation interpolation classes. This extension aligns with the approach of the work conducted by Andersson (2022), but offering a more comprehensive and detailed investigation. In contrast to Lottie animations, the elements constructing a native animation must be declared with platform specific language and classes, resulting in the inability to transfer the created animation to other platforms. In this study the animations created with the native classes and libraries are referred to as the *Native Animation*. While the JSON vector animation is referred to as the *Lottie Animation*.

1.2 Problem Statement

Research regarding the performance of applications developed using cross-platform frameworks is necessary, as the field can be described as immature (Biørn-Hansen et al., 2020). Comparative studies have shown that applications developed with cross-platform frameworks exhibit performance differences when compared to applications developed with a native approach (Biørn-Hansen et al., 2019a; Andersson, 2022; Huber et al., 2020). According to Ciman and Gaggi (2017), the use of cross-platform frameworks for application development always results in a loss of performance when compared to native development. The findings of Biørn-Hansen et al. (2019b) shows that the performance difference is perceived to be a significant challenge of cross-platform mobile applications in the industry, one that can outweigh the higher cost of developing native mobile applications for each platform as well as the inability of sharing code between different implementations (Willocx et al., 2015).

Since the performance difference between native and cross-platform applications depends on the features used by the mobile application (Biørn-Hansen et al., 2019b), it is appropriate to test an application's performance specifically through the use of animations. Animations require high CPU and RAM usage to render re-drawings and state updates of the application's User Interface (UI). The CPU and RAM usage directly affects the performance of mobile devices which are limited by their hardware components (Biørn-Hansen et al., 2019a), highlighting the significance of performance optimization.

Lottie Animations and Native Animations are compared to each other in this study with the aim to answer why mobile applications developed with a cross-platform approach have a higher CPU and RAM usage compared to applications developed using a native approach, and investigate what can be done in order to decrease the performance

difference. Focusing on the performance issues of animations in Flutter applications, this study aims to identify potential solutions to enhance the CPU and RAM usage of cross-platform applications.

It is essential to investigate the performance gap in order to better understand the technical challenges with cross-platform development, increase developers' efficiency and increase the usability of cross-platform solutions. Cross-platform frameworks such as Flutter could become more attractive, effective, less costly and influence developers to choose a cross-platform approach over native development.

1.3 Purpose and Research Questions

The performance gap can influence developers of animation-heavy mobile applications to disregard cross-platform solutions as a viable option for their development, despite the advantages as described earlier. It is thereby important to investigate the performance differences and the trade-offs with using a cross-platform approach compared to a native approach. The result of this study could lead to new knowledge about how the performance could be improved in applications created with cross-platform frameworks. The purpose of this study is therefore:

> To investigate why the CPU and RAM usage differs between applications developed with native technologies and applications developed with a cross-platform framework through the use of two different types of animation implementation. By confirming the performance gap and what factors contribute to it, we aim to propose alternative solutions in order to increase the performance of the cross-platform mobile application during animation execution.

To fulfill the purpose, three research questions were defined. First the performance differences between the applications needed to be confirmed and investigated. Hence our first research question reads as follows.

> RQ1: What are the differences in CPU and RAM usage when handling the Native Animation and the Lottie Animation in mobile applications developed using different approaches, including both native development for iOS and Android, as well as cross-platform development with the Flutter framework?

To be able to provide a possible solution for decreasing the performance differences it is necessary to identify the cause. Hence our second and third research questions read as follows.

> RQ2: Why does an application developed with the Flutter framework have a higher CPU and RAM usage when executing the Native Animation in comparison to the equivalent native application?
>
> RQ3: Why does an application developed with the Flutter framework have a higher CPU and RAM usage when executing the Lottie Animation in comparison to the equivalent native application?

1.4 Scope and Limitations

The limitation of this study was the timeframe in which the study had to be performed. More time would have resulted in a more extensive literature study, which would have led to an increase in the coverage of knowledge about the subject of cross-platform development and more specifically the Flutter framework. The time constraint also affected the quality of the code used for creating the mobile applications. The applications developed were confined to being a *Minimal Viable Product* (MVP) that fulfilled the purpose of the experiments.

The animations were implemented as similar as possible to the industry standard, for each mobile application. Therefore, the Native Animation was implemented with standard native libraries and classes. The use of the libraries and classes transfers the results of the study more appropriately to the industry, making the development more effective which is deemed suitable for this study in regards to the timeframe. Better performance results could be achieved by implementing the animations on a lower level; closer to the hardware components.

Because of the time constraint this study also had to be focused and exclude other cross-platform technologies, features and metrics. Flutter was chosen for this study because of its rapid growth in popularity but including other frameworks and conducting a more extensive comparison study with many features and metrics involved, would further contribute to academia.

The scope of the study's experiments is limited to its technical nature but including user perception of the animations could have been relevant. It could be of interest to study if a user could distinguish between a Lottie and a Native animation across the different

mobile applications. It is possible that including user perception in this study could have affected the discussion section; whether our proposed solution for better performance caters to the user experience or not.

1.5 Disposition

The structure of the ensuing chapters in the thesis report is presented below, where the content of each chapter is explained in more detail.

Chapter 2. *Method and Implementation*: Describes and motivates the development methods employed to create the technical artifact and gather the empirical data, as well as how the literature study was performed, in order to answer the study's research questions.

Chapter 3. *Theoretical Framework*: Consists of relevant knowledge for this study.

Chapter 4. *Results*: Presents the results in the form of analyzed data which was gathered from the experiments.

Chapter 5. *Discussion*: Discusses and describes the connection between the results presented in chapter 4 in regards to the study's research questions, in relation to related studies. Additionally, the chapter also discusses the strengths and weaknesses of the methods chosen to conduct this study.

Chapter 6. *Conclusion*: Contains the conclusions from the conducted study as well as the practical and scientific implications that the results from the study might have. The conclusion chapter also contains suggestions for future research based on the results.

2 Method and Implementation

This study followed a deductive research method where the research gap had been identified through a literature search. To fulfill the study's purpose and answer the research questions, an artifact was developed. The artifact consists of four mobile applications developed with different approaches, i.e. native and cross-platform approach (see chapter 2.2 for detailed information on the artifact development). Each mobile application holds two animation implementations: (1) a Lottie Animation that uses the Lottie API library and (2) a Native Animation built with native libraries provided by the Android SDK, iOS SDK and Flutter framework. Following setup allowed for two experiments that tested the performance of the animation implementations across all developed mobile applications:

1. Lottie Animation Experiment during which the Lottie Animation is tested.
2. Native Animation Experiment during which the Native Animation is tested.

The experiments generated empirical data of mobile applications' performance. Quantitative research was chosen as the most suitable approach for this study since it allowed for a more precise collection of data. To analyze the datasets, a quantitative analysis method was chosen where statistical analysis was applied. The option of conducting qualitative research existed, but was deemed inappropriate for the type of dataset this research would produce.

By measuring and analyzing the CPU and RAM usage of the mobile applications during the Lottie- and Native Animation Experiment, an answer to RQ1 was given. Answers to RQ2 and RQ3 were obtained through analyzing the empirical data in conjunction with the literature study. For RQ2 the results from the Native Animation Experiment were analyzed, while for RQ3 the results from the Lottie Animation Experiment were analyzed. Furthermore, the dataset produced by each mobile application was analyzed for potential performance patterns; comparing the Native Animation against Lottie Animation. By identifying what factors affected the performance gap for each animation implementation and researching how the different OS handle and render animations, the answers for RQ2 and RQ3 were given. This approach contributed to an increased knowledge of potential solutions that could improve CPU and RAM usage for cross-platform solutions.

2.1 Designing the Animation

This chapter provides a more detailed description of the *animation elements* contained in both the Native Animation and the Lottie Animation. In this study an animation

element refers to the individual component that is used to create the visual effect of the animation object's transformation.

As the experiment is highly dependent on the animation used, it was of importance to design a suitable animation of sufficient complexity that would affect the CPU and RAM usage, while also being feasible to create within the given timeframe of this study. The animation that was used in the study of Biørn-Hansen et al. (2019a) was of greater complexity than what could be created within the timeframe, whilst also keeping the visual integrity of the animation. It was thereby considered to be a better practice to design an animation solely for the purpose of this study. It was deemed to be sufficient to capture the same animation elements contained in the *Lottie Star Animation* used in the study of Biørn-Hansen et al. (2019a). By studying the Lottie Star Animation and breaking it down to all its animation elements, an animation for this study could be created to fulfill the purpose as well as producing results comparable to the study of Biørn-Hansen et al. (2019a).

The list below shows all observed animation elements in the Lottie Star Animation that were used to design the Native Animation and the Lottie Animation for the experiments of this study.

- Scaling: Used to change the size of the object.
- Movement: Used to change the position of an object.
- Fading: Used to change the opacity of an object.
- Color change: Used to change the color of an object.

A suitable animation was designed in order to capture all the animation elements. The primary objective of the animation design was to ensure visual consistency across all animation implementations as well as producing comparable code by utilizing equivalent functions and classes in all solutions. The consistency of the animations was ensured by following the tweening instances for each animation element as seen in Appendix A.

Figure 1 shows the stages of the animation design.

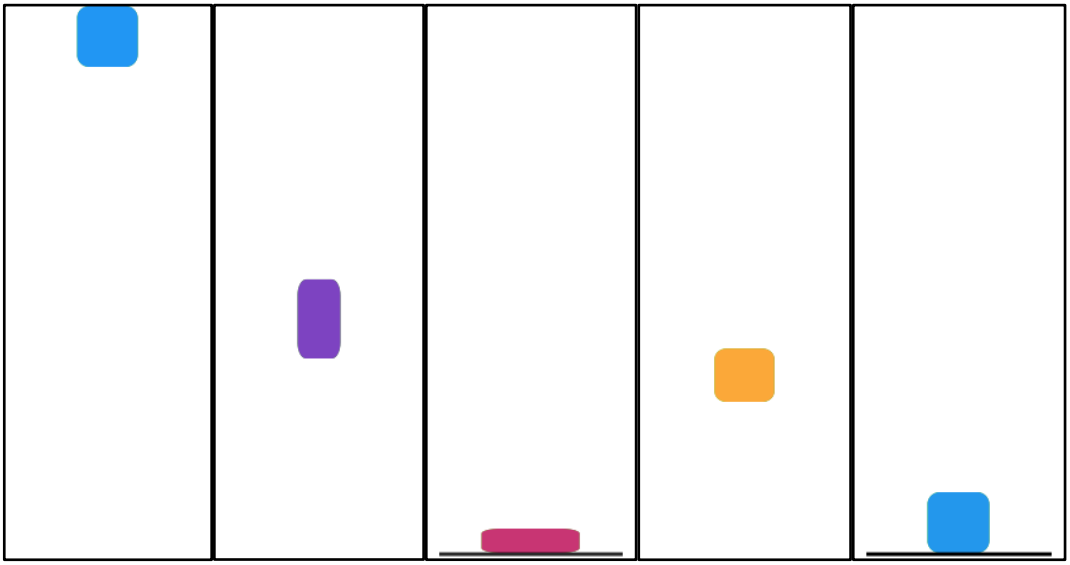

Figure 1: *The flow of this study's animation from start (left) to finish (right).*[1]

The animation contains two objects: a square-shaped box and a line representing the bottom. When the animation is started, the box falls from the top of the screen and bounces back up once it hits the bottom. This process repeats a second time with some loss of velocity where the box falls down again and slightly bounces at the bottom before retaking the initial shape during which time the animation has completed. Each time the box "touches" the bottom, the opacity of the line shifts from transparent to opaque. The illusion of movement is achieved by the animation elements applied to the objects, where the box changes width and height, position and color, and the line changes opacity. The animation has a total runtime of four seconds from start to finish.

2.2 Artifact Development

The technical artifact was created using *Visual Studio Code* for the Flutter framework, *Xcode* for iOS and *Android Studio* for Android, resulting in a total of four mobile applications. Two of which are native mobile applications: one for iOS, developed in Xcode, and one for Android, developed in Android Studio. The other two are the cross-platform mobile applications that were developed using the Flutter framework; a single codebase was developed to be compiled into separate mobile applications for each operating system. A detailed overview of the mobile applications developed can be found in Table 1, which presents the mobile applications based on their development approach and operating system.

[1] The animation can be viewed on LottieFiles at https://lottiefiles.com/144225-falling-box.

Table 1: *Mobile applications by operating system (row) and development approach (column).*

	Cross-Platform	**Native**
Android	Cross-platform Android app	Native Android app
iOS	Cross-platform iOS app	Native iOS app

All mobile applications feature a start screen with two buttons, one for navigating to the Lottie Animation screen, and another for navigating to the Native Animation screen. Both the Lottie Animation and Native Animation screen include a play button to execute their respective animation. The code bases for the developed artifact can be seen in Appendix E.

Figure 2 displays the layout of the start screen, Lottie Animation screen, and Native Animation screen within the mobile applications.

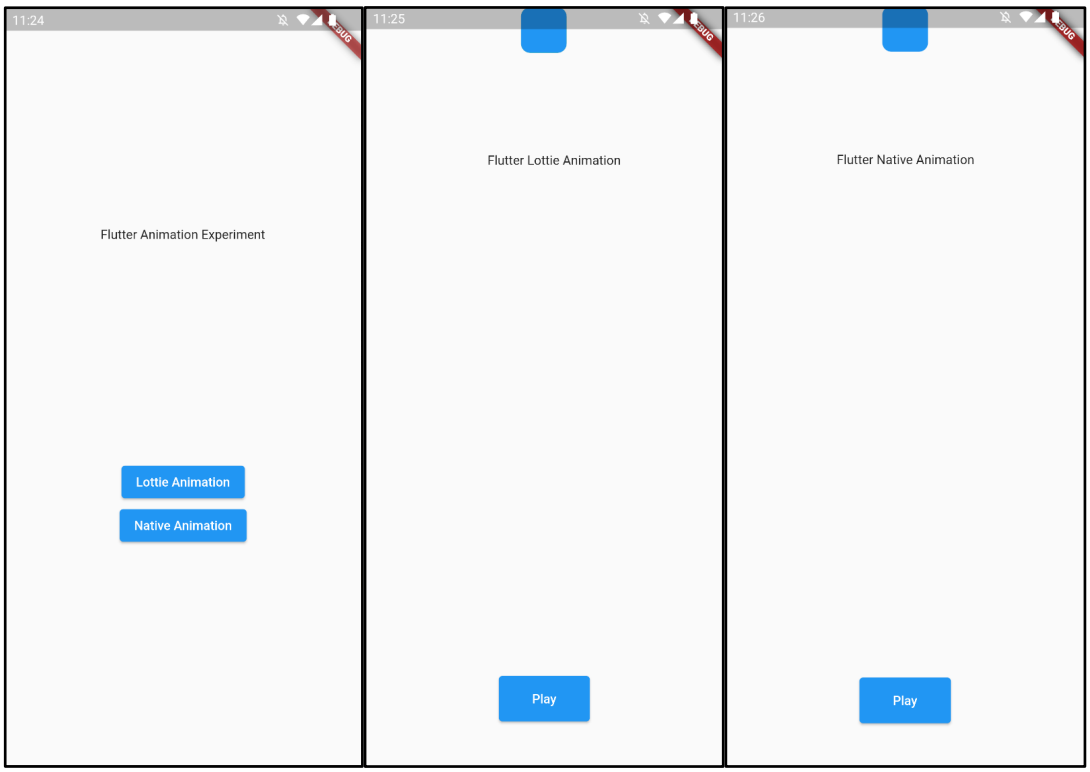

Figure 2: *Screenshots displaying the layout of the start screen, Lottie Animation screen and the Native Animation screen within the mobile applications.*

When the “Play” button is clicked, the animation executes 15 times resulting in a total runtime of one minute (with each animation sequence having a runtime of four seconds).

The focus of the artifact development was to achieve an MVP that utilized the same layout, although the design of the UI components was not customized to look identical across the mobile applications so as to avoid unnecessary code that could affect the experiment results.

2.2.1 Native Animation Implementation

This chapter provides a more detailed description of the Native Animation implementation across each development approach. The Native Animation was created using an implicit animation method which includes classes with the functionality of interpolation between values, also known as *tweening*, resulting in dynamic transformations of the animation objects. Multiple tweening instances were used in order to include the animation elements as described in chapter 2.1. An alternative approach for implementing the Native Animation is to use the explicit animation method. However, this option was rejected due to the potential unnecessary complexity it could introduce.

The following subchapters provide a detailed explanation of the approaches involved in creating the Native Animation for each mobile application.

2.2.1.1 Native Android App

For the development of the native Android app, Android Studio and the *Kotlin* programming language was used. Kotlin was chosen over *Java* since it is considered the primary language for Android development (Google, 2017).

The Android application was developed following the standard practices using Android Studio. The UI layout of the application was created using Extensible Markup Language (XML) files. XML is a declarative file format widely used for defining the structure and presentation of user interfaces in Android applications (Cheon & Chavez, 2021).

An alternative approach could have been used for building the mobile application’s UI, by the use of the UI toolkit *Jetpack Compose*. The use of Jetpack Compose was discarded as a viable option due to the immaturity of the toolkit (Google, 2021), which could have an effect on the stability of the application and possibly the results.

Handling the XML-files is the Kotlin programming language which is most often considered imperative. The combination of these approaches places the Android application somewhat in between a declarative and an imperative approach. The tweening instances in the Android application are declared in an XML file, which allows for a comparable format with the iOS mobile application and the Flutter cross-platform mobile applications. By keeping a similar structure for the tweening instances, the Android native mobile application aligns with the development patterns of the iOS and Flutter mobile applications, making it suitable for the purpose of this study

To create the box and the line in the animation, two separate rectangle objects were created in XML-files within the project's resource files. The tweening of the box's position and scale was implemented through the use of a third XML-file containing the tweening sequence and tweening instances, which were added to the box via the *AnimationUtils* class. Both the color change of the box and the opacity of the line was achieved with the use of the *ValueAnimator* class which interpolated between the given values.

2.2.1.2 Native iOS App

For the development of the native iOS mobile application, Xcode and the *SwiftUI* framework was used. A rectangle object was used in order to create the box and the line as per animation design in chapter 2.1. It was possible to implement animations without the SwiftUi framework. However, it was more suitable to use the framework because of the advantages it offered in terms of code reduction and ease of development which catered to the timeframe of this study. The use of SwiftUI ensured that the Native Animation was comparable with the Flutter and Android mobile applications since SwiftUI also uses a declarative approach.

When selecting the appropriate animation implementation for this study, the framework *Core Animation* was evaluated within the SwiftUI framework. It was possible to use an abstract superclass *CAAnimation* from Core Animation but the solution has an advantage of being specially made to reduce CPU usage, being a deep level integration (Apple, n.d.-b; Apple, n.d.-a). A crucial factor in the decision making process was the necessity of ensuring solution similarity with the cross-platform mobile application and Flutter does not offer the same level of integration for animations. It was therefore decided that using the *withAnimation* function for tweening would be more suitable as it is an abstract function of SwiftUI (Apple, n.d.-i).

The withAnimation function allowed to schedule execution of the tweening instance using the duration and delay parameters. By using withAnimation, only parts of the

view hierarchy that need to be updated for the current animation state are rebuilt. Many tweening instances were implemented to create the animation, each invoking withAnimation function for interpolation between values. The code for the tweening instances had to be sorted manually based on when the function was scheduled to be invoked.

2.2.1.3 Cross-Platform App

The cross-platform mobile application was developed using the Flutter framework (Flutter 3) and the *Dart* programming language. In order to create the box and the line as per animation design in chapter 2.1, a *Container* object is used respectively. The Native Animation was implemented by declaring an instance of the *TweenSequence* class for each animation element. In turn, the TweenSequence consists of multiple tweens, each declared in a *TweenSequenceItem* class. TweenSequence executes each TweenSequenceItem in the order it is declared in the code where the *weight* parameter specifies the duration of the tweening.

In order to add the animation to the Container object, the object needs to be wrapped in an *AnimatedBuilder* widget that automatically rebuilds the part of the widget tree that needs to be updated for the current animation state. This eliminates unnecessary rebuilds of the widget tree; similarly to the native iOS implementation of withAnimation, only parts of the widget tree hierarchy are updated.

2.2.2 Lottie Animation Implementation

The method employed for creating the Lottie animation in this study is based on the official method declared by LottieFiles (n.d.-a).

To create the Lottie Animation, two tools were used: (1) *Adobe XD* for designing the objects -the box and the line- and (2) *Adobe After Effects* for animating the objects. The Adobe tools do not require coding however, similar to working with mobile application views, the dimensions in the Adobe tools are set in pixels, which ensures the same dimensions as those of the Native Animation. Additionally, Adobe After Effects provides the necessary tools for creating the animation elements and timing them accordingly by setting out *keyframes* in the animation timeline view.

To generate an exportable animation in Lottie JSON-format from Adobe After Effects, an extension called *LottieFiles for Adobe After Effects* was used. The animation was then imported into each mobile application in the same format. By importing the Lottie library in the source code, the animation could be parsed and rendered in the mobile

application's interface. Although each mobile application required manual coding to integrate the classes contained within the Lottie library, the method ensured consistency across all mobile applications.

2.3 Data Collection

In order to answer the research questions, it was necessary to gather and analyze data. Two separate experiments were conducted, each recording the performance of the technical artifact while running the different animation implementations, i.e. the Lottie Animation and the Native Animation.

The following subchapters explain the steps taken in order to carry out the experiments and the literature study. Additionally, the chapter also specifies the mobile devices used, the metrics recorded and measuring tools used during the experiment.

2.3.1 The Experiments

Two experiments were conducted in order to gather performance data: (1) Lottie Animation Experiment and (2) Native Animation Experiment. In each experiment the animation implementation was profiled across all four mobile applications, which resulted in a total of eight datasets. The datasets generated from the experiments are presented in Table 2.

Table 2: *Animation performance datasets by mobile application (row) and animation implementation (column).*

	Lottie	**Native**
Native Android	AndroidNative_LottieA	AndroidNative_NativeA
Native iOS	iOSNative_LottieA	iOSNative_NativeA
Cross-platform Android	AndroidFlutter_LottieA	AndroidFlutter_NativeA

	Lottie	**Native**
Cross-platform iOS	iOSFlutter_LottieA	iOSFlutter_NativeA

The animation implementation was executed, looping the animation sequence 15 times, to gather each corresponding dataset. It was necessary to loop the animation in order to produce more precise and accurate datasets. The number of iterations was carefully considered to achieve a balance; looping the animation a sufficient number of times to minimize the impact of background operations on the datasets, while not surpassing a threshold that would lead to collection of more raw data than could be efficiently analyzed within the constraints of this study's resources.

For visualizing the experiment a flowchart was created as seen in Figure 3. The flowchart shows the steps taken during the profiling of each mobile application.

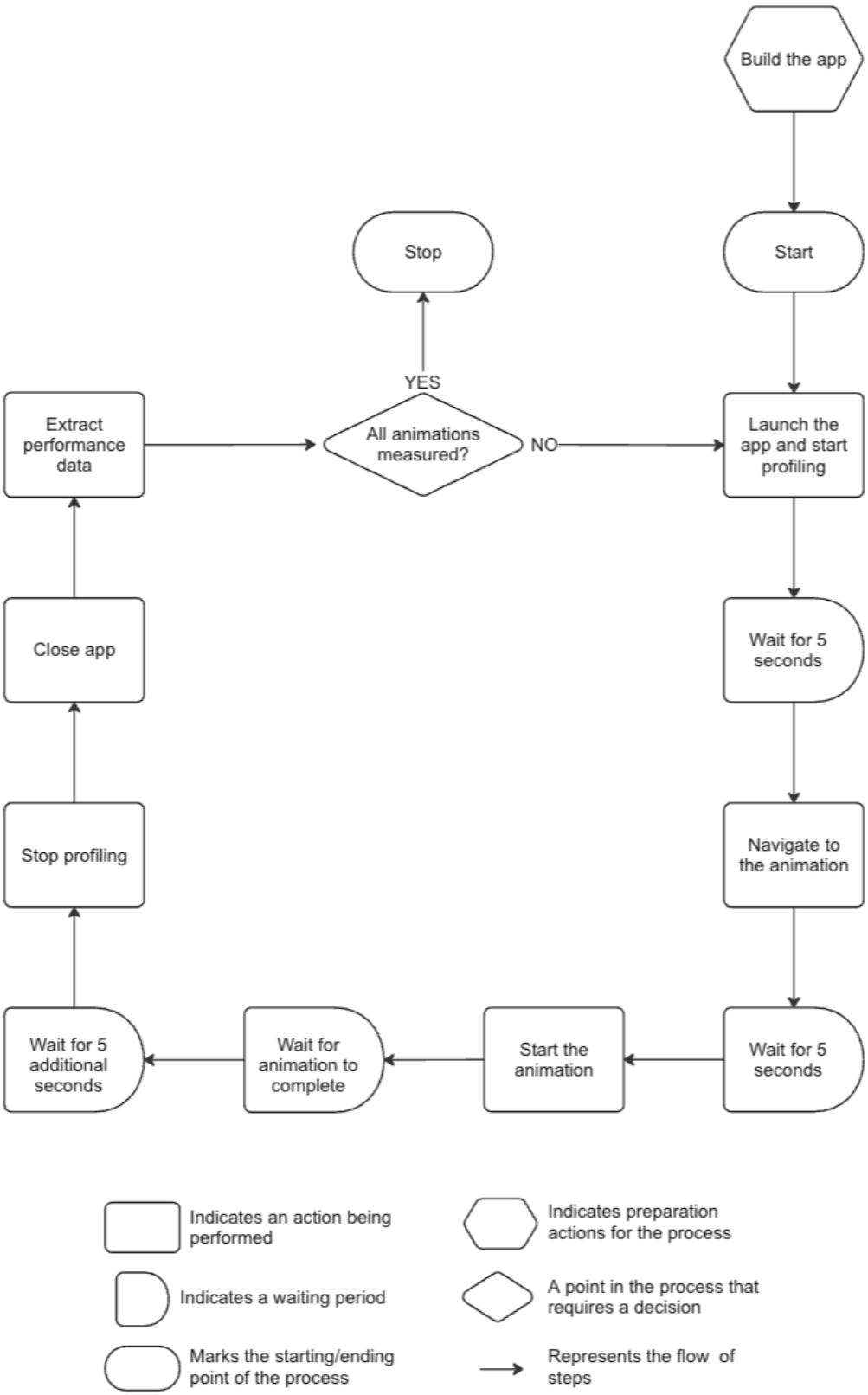

Figure 3: *Flowchart illustrating the profiling process of each mobile application (app).*

Initially before the profiling process begins, the mobile application is built in release mode on the device. In the *Integrated Development Environments* (IDE) the mobile application is built when the device is chosen as the build target. It is crucial to profile the mobile applications in release mode (as opposed to debug mode) to provide more realistic performance data. Profiling an application in release mode allows for a more accurate assessment of its actual performance, as it reflects how it would behave when deployed to the user. In release mode, the application is optimized and stripped of unnecessary debugging information and overhead, resulting in faster execution and more efficient resource utilization.

Once the mobile application is built, the profiling process begins where the profiling mode is launched and set up. The setup procedure for the profiling tool varies between the IDE. However, this does not impact the datasets as the raw performance data is measured and presented in the same format: a graph displaying the CPU or RAM usage over time as seen in Appendix D. In the graphical presentation of performance, the x-axis displays the timeline and the y-axis displays CPU or RAM usage. For more detailed information regarding the profiling tools, refer to chapter 2.3.4.

The nature of raw performance data was taken into consideration which shaped the profiling process. Multiple delays were added in order to isolate the animation performance by allowing the mobile application to fully load after the launch, initiate its state after navigation, and reset its state after the completion of an animation. In addition, the delays improve the readability of the graph by providing a visually distinctive reference point of state changes, i.e. launch, navigation, animation start and animation end.

After stopping the profiling tool, the performance data was extracted. The process was repeated until the performance data for all animations had been successfully collected. Most often, two loops of the profiling process were sufficient for each mobile application, one for each animation. However, in cases of human errors during the process, such as imprecise timing of procedures, additional loops were required. In such cases the corresponding graph was discarded and the process was repeated to ensure validity of the datasets.

2.3.2 Literature Study

Four articles within the field of cross-platform mobile application performance were initially found which served to define the research gap and the research questions. The articles used for the foundation of this study were: (1) Biørn-Hansen et al. (2019a), (2) Andersson (2022), (3) Nawrocki et al. (2021) and (4) Huber et al. (2020). The purpose

and contribution of these articles can be seen in Appendix B, Table B1, while their research and findings are described in detail in chapter 1.

With the research gap defined, additional articles were found through the use of the searching tool *Google Scholar* and by searching the databases *Primo* and *Diva* for scientific articles on the subject of this study. The searches consisted of keywords that were deemed suitable for finding articles in the same field of knowledge. The keywords used were: *Cross-platform, Flutter, mobile, software, development* and *performance.* A combination of the keywords was also used to yield the most suitable results.

For defining the research gap the focus was primarily on comparative studies not older than five years. The five year limit ensured that the scientific literature included was relevant, considering the rapid evolution of technologies in the mobile application development domain. However, an exception was made for literature addressing foundational concepts in mobile application development, animation techniques and performance metrics. The reason being that these concepts are more stable over time, providing valuable insights regardless of the publication date. Hence, resources that were older than five years were also included in this study for that particular reason.

From an initial pool of articles, 16 articles were selected based on their relevance to the research questions and the scope of this study. Each article was examined based on the information presented in the abstract. Focusing on the potential contribution these studies could have for the technical investigation of this study. Studies focusing on user perception or user experience were excluded as their findings were not considered useful for this study. Comparative studies or benchmarking studies were most useful as the information and findings from those were technical and detailed. A full list of the articles used in this study can be seen in Appendix B, Table B1.

In addition to the academic articles, information was also gathered from 36 websites to understand the technology and tools used. These websites can be seen in Appendix B, Table B2.

2.3.3 Technologies

This study utilized multiple technologies to create the mobile applications. Table 3 displays the versions, approaches, programming languages, and IDE used for each technology.

Table 3: An overview of the technologies used.

Technology	Version	Approach	Language	IDE
Native Android	SDK 32 Lottie 3.4.0	Native	Kotlin	Android Studio
Native iOS	Xcode 14.2 Swift 5.7 Lottie 4.2.0	Native	Swift	Xcode
Flutter Framework	Flutter 3.7.3 Dart 3.0.0 Lottie 2.3.0	Cross-compiled	Dart	Android Studio Xcode

2.3.4 Metrics and Measuring Tools

During the experiments mentioned in chapter 2.3.1, the performance of each mobile application has been scrutinized and the following list of metrics have been recorded:

- RAM usage, measured in *megabytes* (MB).
- CPU usage, measured in percentage (%).

Figure 3 provides a visualization of the steps taken during the experiments to capture the CPU and RAM usage for each mobile application and animation implementation. Both the CPU and RAM usage were measured during the animation execution which captured the real-time performance. Additionally, the RAM usage was measured pre- and post-animation execution. Measuring the RAM usage during the different stages provided a more comprehensive understanding of the performance of the mobile applications. By measuring the RAM usage pre-animation and during the animation, an insight could be gained into the amount of additional memory required to execute the animation. Measuring RAM usage post-animation will show how the mobile application performs while in an idle state and whether the mobile application releases any memory.

The performance measurements were taken using the native IDEs of Android Studio and Xcode. The cross-platform mobile application is compiled to the native mobile operating system of the device it runs on, which is why the same tools used for measuring the performance of the native mobile applications were used for the cross-platform mobile application as well. Reusing the tools in such a way is beneficial for producing more comparable datasets; if the measuring tool had an impact on the

measurement of a native mobile application, the same could be assumed for the cross-platform application. The tools used for measuring the CPU and RAM usage on the Android and iOS platforms are displayed in Table 4.

Table 4: *Tools for measuring the RAM and CPU usage for the Android and iOS platforms.*

Metric	**Android Measuring Tool**	**iOS Measuring Tool**
RAM	Android Studio Profiler	Instruments: Allocations
CPU %	Android Studio Profiler	Instruments: Time Profiler

It is worth mentioning that while there are third-party measuring tools available for measuring CPU and RAM usage (which could potentially make it possible to use one tool for all platforms), it was decided not to use them in this study. This decision was based on findings from Biørn-Hansen et al. (2019a) that suggests that such tools may have an impact on the accuracy of the results.

2.3.5 Devices

Two mobile devices were used during the study: an Android device for running the Android mobile applications, and an iOS device for running the iOS mobile applications. Information about the mobile devices used in this study are listed in Table

Table 5: *Display of the different mobile devices that were used in the study.*

Mobile Device	Samsung Galaxy S20 FE 5G	iPhone 8
Operating system	Android version 13	iOS version 16.4.1
Processor	2,8GHz octa-core	2,4GHz hexa-core
Memory	6 GB	2 GB

The devices used in the experiments were factory reset in order for the OS to have the optimal functionality without any background tasks not relevant to the experiments.

2.4 Data Analysis

The experiments produced eight datasets: each accounting for the performance of an animation implementation in one of the mobile applications. The raw empirical data

consisted of CPU and RAM usage plotted across a timeline. A limitation of the profiling tools used is that they did not provide an indication of when the animation starts and ends, as well as where a new animation loop begins. To address this limitation, the empirical data was manually cleaned by identifying the time-points when the animation loop had started and exited. Additionally, the predetermined duration of animations was used as a reference to identify each loop iteration. By closely monitoring these time-points, the performance could be accurately extracted for further analysis.

When extracting the performance data, an interval of one second between each recorded data point was noted in an Excel-sheet, resulting in 60 data points for each dataset. An exception of the one second interval of the data extraction was made for the Native Animation in the native iOS mobile application, where the same method could not be employed because of idle states occurring between the animation iterations. Instead the performance was measured four times for each animation iteration as the idle states made it obvious where a new iteration began. The adjustment ensured that the performance data was exclusively extracted during the active execution of the animation, thereby excluding the idle states. This strategy was essential to avoid the recording of false data points that would create a misleading dataset.

Additionally, recording of performance data pre- and post-animation were taken. To ensure that the mobile application's performance was accurately represented at these stages, each state was given a period of five seconds to stabilize during the experiment. This meant that the mobile applications were allowed to settle into a state of rest, ensuring that the data captured would accurately represent the stages.

For data analysis, descriptive statistics were applied to the datasets. The mean CPU and RAM usage was calculated to provide a representation of the overall performance during the execution of animations. The peak CPU and RAM usage was recorded to represent the “worst case scenario”. This was achieved by manually pinpointing the highest values in the graph during the animation execution.

2.5 Validity and Reliability

To keep the validity and reliability of the experiments high, the mobile applications developed were reduced to a MVP. This decision was based on minimizing the differences between the mobile applications, with the primary distinction being the development approach and animation implementations employed. This isolates the performance measurement to being dependent on the underlying operating system, thereby enhancing the validity of the study.

Considerable effort was invested in ensuring that the animation implementations were similar across the mobile applications, with the cross-platform solution serving as a guideline. However, this approach has the drawback of disregarding other (potentially better) solutions that exist in native approaches, which would offer more optimized code. This is due to the native approach having a deeper level of integration whereas the Flutter framework adds a layer of abstraction. Ultimately, the comparability of the empirical data was prioritized in order to fulfill the purpose of this study. The native approach required a more abstract solution to cater a fair comparison.

Another challenge encountered was achieving visual consistency of the animations as none of the solutions, both regarding the Native Animation and Lottie Animation, were easily transferable in terms of timing. That was especially apparent when implementing the Native Animation in native iOS mobile applications. A limitation of the SwiftUI framework was the absence of a built-in method to sort the tween sequences in the correct order. While SwiftUI is a declarative framework, in this particular scenario, it was unable to accurately sequence the tweening to achieve the desired timing. That was potentially caused due to the extensive amount of tweening implemented.

In an effort to preserve the validity of the experiments conducted, several precautions were taken. To reduce external influence on the datasets, all non-essential services were turned off so as to not put any further load on the devices. Bluetooth, location services, and Wi-Fi remained turned off during the course of the experiments. Additionally, a consistent environment was established on all mobile devices, keeping the battery and the brightness level of the screen at 100%.

2.6 Considerations

By making cross-platform development more attractive to developers, time and money can be saved due to the use of a single codebase. This can result in a more sustainable industry where companies choose to use a cross-platform framework for their development and as such saving resources in the form of human employment-, server- and electrical costs.

To ensure unbiased data collection, 15 animation iterations were conducted during the mobile application experiments. This approach aimed to provide more precise mean values. However, it is important to note that an excessive number of iterations could result in irregular measurements caused by factors beyond the scope of this study.

Another important consideration is the environment in which the applications were tested. In this study, the performance profiling was conducted on devices that had been

factory reset. While this offers a clean and uniform basis for comparison, it may not accurately represent the conditions of “real world” usage. Such devices contain numerous additional applications, background processes, and varying battery levels. All of which can influence the performance of a mobile application.

3 Theoretical Framework

This chapter provides a foundation of the concepts, tools and technologies associated with mobile application development. This overview includes a wide range of topics, ranging from native and cross-platform development to specific technologies and platforms like Android, iOS, and Flutter.

3.1 Native Development

When developing a mobile application natively the developers use the *Software Development Kit* (SDK) specific to the targeted mobile OS (Menegassi & Endo, 2020). This approach allows the mobile application's features and functions to be tailored to the native platform's mobile operating system, which enables the application to access the full range of the device's features (Vilcek & Jakopec, 2017). Tools for building and debugging native applications are provided by the native platform IDE (Xanthopoulos & Xinogalos, 2013). The native platforms are restricted to their specific programming languages of the platform IDE. Where the Android IDE supports the languages Java, Kotlin, and C++, while the iOS IDE supports Objective-C and Swift (Biørn-Hansen et al., 2020).

Drawbacks of native development are the inability to reuse the source code for other platforms than the native one. This leads to an increased development cost if an application is to be present on multiple platforms, since the development needs to be re-performed for every platform (Barros et al., 2020). In contrast to the drawback of the inability to reuse code across different platforms, native development has a substantial performance advantage regarding the CPU and RAM usage in comparison to applications developed with a mobile cross-platform approach (Huber et al., 2020).

3.1.1 Android

Android is a mobile operating system developed by Google. Android mobile applications are developed with the use of the native IDE of Android, Android Studio. The IDE contains tools for code editing, debugging, and visual design. Android applications are developed with the imperative programming languages Java or Kotlin. The official language of Android Studio was initially Java, which is still supported on the platform, but since 2017 Kotlin is the primary language used for Android development which was announced by Google in 2017 (Google, 2017).

Android applications are further developed with the Android SDK which provides native libraries and APIs allowing for effortless application integration with the

Android platform. The Android UI uses a declaratively approach where the layout is defined in XML-files in the project's resources files (Cheon & Chavez, 2021).

For rendering the UI, Android uses the Skia Graphics Engine which is an open source 2D graphics library that delivers common APIs operable across a variety of hardware and software platforms (Skia, n.d.). Sponsored and managed by Google, Skia Graphics Engine is not only used by Android but also renders the graphics for various other platforms (Skia, n.d.). Notably, Skia Graphics Engine also renders the graphics in Flutter (Skia, n.d.), which is discussed further in chapter 3.3.

3.1.2 iOS

iOS is a mobile operating system developed by Apple Inc. that powers their iPhone, iPad, and Mac devices. iOS mobile applications are developed with the use of the native IDE of iOS, Xcode (Apple, n.d.-j). Xcode provides a complete set of tools for writing, compiling, and debugging code, as well as for designing and building user interfaces (Apple, n.d.-j). The programming language used in Xcode is the imperative language *Swift* (Apple, n.d.-j). The iOS SDK provides the software libraries and frameworks necessary for building iOS mobile applications (Apple, n.d.-f).

Some of the core frameworks used for iOS development include *UIKit* which provides the basic building blocks for iOS mobile application interfaces (Apple, n.d.-h), *Foundation* which provides essential classes and utilities for developing iOS mobile applications (Apple, n.d.-e), *Core Data* which provides data storage and management functionality (Apple, n.d.-d), and Core Animation which provides powerful graphics and animation features (Apple, n.d.-c).

The SwiftUI framework provides an additional way of building the UI in the mobile application. SwiftUI employs a minimalistic coding approach with a declarative syntax (Apple, n.d.-g). It is possible to state that a UI should display a list of items with text fields, and subsequently specify the alignment, font, and color for each field (Apple, n.d.-g). This straightforward way of declaring UI results in a simpler and easily understandable code, reducing both development time and maintenance effort (Apple, n.d.-g). Moreover, the declarative style of SwiftUI simplifies the implementation animations by managing runtime operations for movements with only a few lines of code (Apple, n.d.-g). Although SwiftUI offers tools for building an animation, it still relies on Core Animation for rendering the graphics. As explained by Hudson (2021), SwiftUI uses Core Animation for its rendering by default.

3.2 Cross-Platform Development

Code sharing between native applications exists, but it is impractical (Biørn-Hansen et al., 2019b). Cross-platform development solves the issue of having to write code for each platform more efficiently, with the approach of "Write once run anywhere" (Huber et al., 2020), where a single codebase can be used for multiple platforms (Biørn-Hansen et al., 2020).

There are multiple approaches for cross-platform development. Ciman and Gaggi (2017) mention four main approaches; *Progressive Web Applications* (PWA), *Hybrid Applications*, *Runtime-based and Interpreted Applications* and *compilation-based Applications*. Biørn-Hansen et al. (2020) extends Ciman's and Gaggi's (2017) four approaches by including *Model-driven software development* as a fifth approach.

The five cross-platform development approaches and popular framework that uses the approaches can be seen in Table 6.

Table 6: *A description of the five cross-platform development approaches and the most popular frameworks using the approaches.*

Cross-platform development approach	Description	Development Tools and Technologies
Progressive Web Apps	Uses HTML, CSS and Javascript to develop a web application that can be accessed through a web browser on a mobile device (Ciman & Gaggi, 2017). The application cannot be installed directly on the mobile device and is hence dependent on the platform's web browser to run. A drawback with PWA is that the application cannot access features on the platform of the device if the feature is not exposed through the web browser (Biørn-Hansen et al., 2020).	AngularJS, jQuery, Svelte, React.js, Vue.js
Hybrid apps	The application created with the hybrid approach packages HTML, CSS, and JavaScript files into an application that can be installed on the mobile device. The application	Apache Cordova, Framework7, Ionic, Quasar,

	behaves like a native application with access to native functionality, and can be downloaded from the app stores and used without the need of an internet connection (Biørn-Hansen et al., 2020).	
Runtime-based and Interpreted apps	Runtime-based and Interpreted apps make it possible to use different programming languages for the application than the native language of the platform where the application is running. To convert and execute the written code to the native programming language an interpreter is also installed on the device with the application. The native features of the platform are accessed through an abstraction layer, which generally affects the performance of the application negatively (Ciman & Gaggi, 2017).	NativeScript, ReactNative, Xamarin,
Compilation-based apps	Similarly to the Runtime-based and Interpreted apps approach, the Cross-Compiled approach makes it possible for a developer to use a common programming language for creating an application running on different platforms. With the difference of the application not needing the use of an interpreter for compiling the code to native code. Instead, the framework generates different native applications which are using the native language of the platform (Ciman & Gaggi, 2017; Biørn-Hansen et al., 2020).	Flutter, Mono, MoSync
Model-driven software development	Model-driven software development uses a higher level of abstraction than the source code for the development of a cross-platform application. The most often used models for this approach are Domain Specific Languages (DSL) or General-purpose modeling (GPM) such as the use of Unified Modeling Language (UML). Through a generator the code is transformed into source code for the platform used, resulting in an application that can be	MAML, MD2, Applause

	identical to a native application (Biørn-Hansen et al., 2020).	

A disadvantage of applications built using cross-platform approaches is their performance, which puts a higher strain on the mobile device CPU and RAM compared to native applications which are suited to a specific platform (Huber et al., 2020). Cross-platform applications having less extensive integration with the mobile device and its native APIs compared to their native counterparts (Shah et al., 2019). Another disadvantage of cross-platform applications is the challenge of testing the applications, due to there being a wide variety of different mobile devices, each with distinct configurations and hardware that needs to be catered to (Menegassi & Endo, 2020).

3.3 Flutter

Flutter is an open-source, compilation-based framework that compiles natively for Android and iOS (Biørn-Hansen et al. 2020) in order to build multi-platform applications from a single codebase.

Flutter is a declarative framework that builds the UI based upon the current state of the application (Flutter, n.d.-c). Widgets are used to render the UI components in Flutter, which are natively compiled, eliminating the need of using and rendering native UI components. Two sets of widgets are included in the Flutter framework; *Material design* widgets used by Android and *Cupertino design* widget used by iOS (Cheon & Chavez, 2021). The use of widgets provides views and functions for facilitating the reading and displaying of the applications (Faiz et al., 2022), and together form a widget tree where one widget leads to another.

Flutter code is written in the Dart programming language, which was released by Google in 2011 (Sharma et al., 2022). The code is compiled with just-in-time (JIT) compilation during the development process to support hot reloads and debug mode. When the application is in release mode, the framework switches to ahead-of-time (AOT) compilation to compile the high-level programming language Dart to native 32-bit and 64-bit ARM code for iOS and Android (Boukhary & Colmenares, 2019) before the execution of a program or a function, which reduces the workload at run time.

For rendering the graphics on Android devices, Flutter uses Google's Skia Graphics Engine which eliminates the need of native UI elements (Nawrocki et al., 2021). Skia is an open-source graphic library that provides a common set of APIs for mobile platforms (Andersson & Axelsson, 2021). For rendering the UI for the iOS platform

Flutter uses the Impeller engine (Flutter, n.d.-a) which was released in Flutter 3.7 (Flutter Team, 2023).

3.4 Lottie Animation

LottieFiles is a library for vector-based animations that can be shipped and rendered in both native and cross-platform mobile applications. The animations in the library are called Lotties, stored in a small JSON-file that achieves the same quality as the GIF and MP4 format. The elements contained in the Lottie animation are exposed which provides the ability for runtime interaction with the elements in applications (LottieFiles, n.d.-d).

The Lottie animations are created with motion graphics tools, which can export the animations in the desired format. Lottie animations requires the Lottie library to be imported in an application to execute the animations via either URL or local folders. Lottie provides their animation libraries across a broad range of mobile and web platforms, allowing the same animation to be used for multiple platforms (LottieFiles, n.d.-b).

3.5 Adobe After Effects

Adobe After Effects is a tool for motion graphics and visual effects. With Adobe After Effects animations in 2D and 3D can be generated for film, tv, video, web or mobile applications. Adobe After Effects adds motion to graphics by keyframes, ranging from simple animations to building 3D worlds. Keyframes are specific points in time where a parameter or property, like position, scale, or color, is set. These keyframes define the start and end points of any transition or change in the animation. Additionally, Adobe After effects compositing tools enable the combination of multiple layers into the same animation (Scott, 2017).

Adobe After Effects is also the recommended motion graphics tool for creating Lottie animations. Which can be exported in JSON-format directly to LottieFiles from the Adobe After Effects client through a Lottie plug-in (LottieFiles, n.d.-a).

4 Results

The findings from the performance measurements of the CPU and RAM usage in the Native Animation experiment and the Lottie Animation experiment are displayed in the following subchapters where the mean and the peak usage from the recorded data points are displayed as well as the pre- and post- RAM usage. The complete datasets with the recorded data can be seen in Appendix C.

4.1 Native Animation Analysis

The findings from the Native Animation experiment can be seen in Table 7 which displays the mean and peak CPU usage of the Native Animation datasets and in Table 8 which displays the mean and peak RAM usage of the Native Animation datasets.

Table 7: *CPU usage results from Native Animation performance experiment for each mobile device.*

Mobile Device *Dataset*	**Mean CPU Usage (%)**	**Peak CPU Usage (%)**
Android Device AndroidNative_NativeA AndroidFlutter_NativeA	 3.9 6.1	 5.0 8.0
iOS Device iOSNative_NativeA iOSFlutter_NativeA	 30.3 55.2	 100.0 77.1

Table 8: *RAM usage results from Native Animation performance experiment for each mobile device.*

Mobile Device *Dataset*	**Pre RAM Usage (MB)**	**Post RAM Usage (MB)**	**Mean RAM Usage (MB)**	**Peak RAM Usage (MB)**
Android Device AndroidNative_NativeA AndroidFlutter_NativeA	 59.0 140.9	 71.0 141.6	 72.8 141.8	 75.6 146.2
iOS Device iOSNative_NativeA iOSFlutter_NativeA	 5.6 28.6	 6.1 29.7	 6.1 29.7	 6.2 29.8

4.2 Lottie Animation Analysis

The findings from the Lottie Animation experiment can be seen in Table 9 which displays the mean and peak CPU usage of the Lottie Animation datasets and in Table 10 which displays the mean and peak RAM usage of the Lottie Animation datasets.

Table 9: *CPU usage results from Lottie Animation performance experiment for each mobile device.*

Mobile Device *Dataset*	**Mean CPU Usage (%)**	**Peak CPU Usage (%)**
Android Device AndroidNative_LottieA AndroidFlutter_LottieA	 5.6 5.5	 9.0 9.0
iOS Device iOSNative_LottieA iOSFlutter_LottieA	 2.4 64.0	 82.5 100.0

Table 10: *RAM usage results from Lottie Animation performance experiment for each mobile device.*

Mobile Device *Dataset*	**Pre RAM Usage (MB)**	**Post RAM Usage (MB)**	**Mean RAM Usage (MB)**	**Peak RAM Usage (MB)**
Android Device AndroidNative_LottieA AndroidFlutter_LottieA	 70.7 150.4	 76.5 149.6	 74.2 151.1	 76.5 154.9
iOS Device iOSNative_LottieA iOSFlutter_LottieA	 5.7 27.9	 5.6 30.0	 5.6 30.0	 5.7 30.1

5 Discussion

In this chapter, a discussion is presented for each research question, aiming to provide answers and insights that emerged from the findings of this study. The strengths and weaknesses of the methodology used is also discussed at the end of the chapter.

5.1 Result and Analysis Discussion

The purpose of the study which was stated in chapter 1.2 was to confirm and investigate why the CPU and RAM usage differs between applications developed with a cross-platform approach and a native approach, through the use of animations.

The results from the Native Animation experiment and the Lottie Animation experiment both clearly showed that cross-platform applications and both of its animation implementations performed worse than the native applications in regards to the CPU and RAM usage. The exception being the Android cross-platform Lottie Animation which performed almost identical to the Android native Lottie Animation in terms of CPU usage.

5.1.1 Research Question 1

The first research question aimed to identify the differences in CPU and RAM usage between the Native and Lottie Animations across the different mobile application development approaches:

> RQ1: What are the differences in CPU and RAM usage when handling the Native Animation and the Lottie Animation in mobile applications developed using different approaches, including both native development for iOS and Android, as well as cross-platform development with the Flutter framework?

The results from the animation experiments were analyzed and provided an answer to RQ1.

Although the results are discussed in order of decreasing performance, note that they highlight the differences in CPU and RAM usage between native and cross-platform mobile applications. Native iOS and Android mobile applications are not comparable to each other since different operating systems have different performance characteristics and resource management techniques which makes direct comparisons misleading. Therefore, the aim of RQ1 is to present how cross-platform mobile

applications perform in comparison to their native counterparts within the same operating system.

Regarding the Native Animation Experiment, the following datasets were analyzed:

- AndroidNative_NativeA
- AndroidFlutter_NativeA
- iOSNative_NativeA
- iOSFlutter_NativeA

From the results it was found that the mean CPU usage was lowest in the Android Native with 3.9%, followed by Android Flutter with 6.1%. Both iOS Native and iOS Flutter had significantly higher CPU usages at 30.3% and 55.2% respectively. Peak CPU usage was lowest in Android Native at 5%, followed by Android Flutter at 8%, iOS Flutter slightly lower at 77.1% and iOS Native reaching 100%.

Regarding the RAM usage during the Native Animation execution, it was found that mean RAM usage was the lowest in iOS Native with 6.1MB, followed by iOS Flutter with 29.7MB. Both Android Native and Android Flutter had higher mean RAM usage at 72.8MB and 141.8MB. Following the same order, the peak RAM usage was lowest in iOS Native with 6.2MB, followed by iOS Flutter with 29.8MB, Android Native with 75.6MB and Android Flutter with 146.2MB.

As for the Lottie Animation, the following datasets were analyzed:

- AndroidNative_LottieA
- AndroidFlutter_LottieA
- iOSNative_LottieA
- iOSFlutter_LottieA

From the results it was found that the mean CPU usage was lowest in the iOS Native at 2.4%, followed by Android Flutter and Android Native (which had nearly identical mean CPU usage) at 5.5% and 5.6% respectively. The highest mean CPU usage was recorded in iOS Flutter at 64%. Peak CPU usage was lowest in Android Native and Android Flutter which both recorded a peak of 9%, followed by iOS Native at 82.5% and iOS Flutter at 100%.

Regarding the RAM usage during the Lottie Animation Experiment, it was found that mean RAM usage was the lowest in iOS Native with 5.6MB, followed by iOS Flutter with 30MB, Android Native with 74.2MB and Android Flutter with 151.1MB. Following the same order, the peak RAM usage was lowest in iOS Native with 5.7MB,

followed by iOS Flutter with 30.1MB, Android Native with 76.5MB and Android Flutter with 154.9MB.

The increased RAM usage of the Flutter mobile applications could be due to Flutter's runtime architecture, which requires the loading of the entire library. Consequently, even for less demanding processes, resources for the entire library must be allocated. This can result in higher RAM usage. However, this particular reason might not be the only factor contributing to the differences in RAM usage observed. Other aspects such as different programming practices, implementations, and device specific factors could also influence RAM usage.

It is important to mention the pre animation RAM usage when analyzing the result for both animations. Android mobile applications had a higher RAM usage overall; here the pre value becomes important as it exposes the true memory allocation of the animations. Looking at the relative increase of the mean RAM usage, from pre- to during animation, it becomes clear that the performance is more similar across all mobile applications where the mean RAM usage is generally increasing by 1-4MB. An exception is the Android Native mobile application within the Native Animation Experiment, which displayed a higher increase of approximately 13MB. This could be due to the way XML-files are parsed by the Android SDK. When the app is compiled, the Android SDK parses the XML-files into a binary format which is inflated at runtime to create view objects representing the UI elements (Android, n.d.-a) . When the XML-files have been inflated into view objects, they can interact with the rest of the Android SDK (Android, n.d.-d).

In regard to the Native Animation Experiment, the Flutter mobile application exhibited worse performance in both CPU and RAM usage on both the Android and the iOS device. The findings of Andersson (2022) displayed the same pattern as our findings of both the cross-platform and the native mobile application's performance on the Android device. The results also aligned well with the work of Huber (2020) which found the cross-platform mobile applications to perform worse than the native Android mobile application on all tested Android devices. The RAM usage was also higher for the iOS Flutter mobile application, which corresponds with the findings of Nawrocki et al. (2021) where the RAM usage was consistently higher in the Flutter mobile application than in the native iOS mobile application. The iOS Flutter mobile application displayed significantly higher CPU usage than the iOS native mobile application which is not as severe as in the study of Nawrocki et al. (2021). However, their study did not examine animations which could be a possible reason.

Our findings align well with the study of Biørn-Hansen et al. (2019a) which found that cross-platform mobile applications resulted in a higher RAM usage when executing Lottie animations. The CPU usage in the iOS device also matched the findings of Biørn-Hansen et al. (2019a). However, the CPU usage trends for the Android device in our study demonstrated a more homogeneous result in comparison to the cross-platform frameworks examined in their research (which did not include Flutter). In the Lottie Animation Experiment, the Android Flutter mobile application had an almost identical CPU usage as the Android Native mobile application. This result is not completely surprising since the study of Nawrocki et al. (2021) found that the Flutter mobile application was the only application to beat the native Android mobile application in terms of CPU usage. Then again, their study did not test the CPU usage through animations which could have yielded a different result.

Overall, our results revealed significantly higher CPU and RAM usage in cross-platform mobile applications compared to the native counterpart, particularly in the iOS compiled version. Interestingly, the Android Flutter mobile application maintained comparable performance to Android Native mobile application when executing the Lottie Animation.

5.1.2 Research Question 2

The second research question aimed to understand why the cross-platform mobile application developed with Flutter exhibits higher CPU and RAM usage during the Native Animation execution compared to the native mobile applications:

> RQ2: Why does an application developed with the Flutter framework have a higher CPU and RAM usage when executing the Native Animation in comparison to the equivalent native application?

Based on our findings in RQ1, we can speculate about the reasons behind the cross-platform mobile application's performance behavior.

Flutter relies on a declarative approach to build the UI, representing the interface as a direct outcome of the application's current state rather than modifying UI components incrementally as with an imperative approach (Flutter, n.d.-c). When a widget's state changes, the widget rebuilds its description, which is then used by the Flutter framework to determine the minimal changes needed in the underlying widget tree (Flutter, n.d.-b). SwiftUI framework used for native iOS development also employs a declarative UI approach (Apple, n.d.-g), while the native Android solution uses both

declarative and imperative, which is an indication that other factors contribute to the observed differences in CPU usage. Flutter widgets, while being compiled into native code that runs on the specific platform, are not the same as the native UI components provided by the iOS and Android SDK. Unlike native UI development, Flutter constructs its own UI using these widgets which are then rendered by a graphics engine.

Since the release of Flutter 3.7, iOS is by default using a new graphics engine named Impeller which is developed specifically for Flutter rendering (Flutter Team, 2023). Impeller engine addresses the performance issues of the Skia Graphics Engine (that is not developed specifically for Flutter) by enabling animations to be captured and persisted to disk without impacting rendering performance (GitHub, n.d.-b).

Android continues to use the Skia Graphics Engine by default since the Impeller engine is not yet ready for preview on Android devices (Flutter, n.d.-a). With the Impeller engine being immature at this time, very limited information regarding the engine could be found. In essence, the Impeller engine aims to increase the performance of rendering of the Flutter framework. Although the engine focuses more on improving the visual experience of UI by addressing the issues with janky frames (Flutter, n.d.-a). Nonetheless, according to our results the cross-platform approach performed better on Android than iOS.

Whether the Skia Graphics Engine or the Impeller engine is used for the cross-platform mobile application, additional demand is put on the CPU as the animations are abstracted and therefore require further processing resources for rendering. Both iOS and Android have a low level layer for graphics rendering on the canvas that the Flutter framework must access. Core Animation is used for rendering the animations on the iOS mobile applications, regardless of the animation being implemented in SwiftUI (Apple, n.d.-c). This framework optimizes the animation process by making use of CALayer, a class that manages image-based content and draws animations on the canvas (Apple, n.d.-b).

Similarly on Android, the Canvas class is used for drawing on the canvas (Android, n.d.-c). These native mobile applications have an inherent advantage as they are deeply integrated and optimized for the respective operating systems. In contrast, Flutter widgets have a disadvantage of an additional layer of abstraction which makes it more difficult to access the lower layer effectively. Again, the performance gap between Native Android and Flutter Android is very small. This can be attributed to a closer cooperation between the developer teams (as both are from Google), which led to a more successful integration with the lower level. The iOS and Flutter being developed independently from each other, could contribute to a bigger performance gap.

As mentioned in chapter 5.1.1, the RAM usage of the Flutter mobile application may be attributed to Flutter's runtime architecture. An additional factor possibly contributing to the high memory consumption of the cross-platform mobile application is the Dart garbage collector used by Flutter. The Dart garbage collector actively manages memory allocation and deallocation through Generational garbage collection. With this garbage collection approach, memory is divided into several "generations" where the garbage collector primarily focuses on the young generation. Objects that survive collection in the young generation are eventually moved to the old generation (Flutter Team, 2019). In Flutter widgets are created as they are rendered, destroyed or rebuilt on state changes or when they are no longer visible on the screen (Flutter Team, 2019). The Native Animation utilizes multiple widgets with constant state changes that the Dart garbage collector must keep track of, which can impact RAM usage.

The three technologies examined in this study all have different approaches to their memory allocation and deallocation. iOS does not use a traditional garbage collector but instead uses Automatic Reference Counting (ARC). ARC keeps a count of the references for each object, if the count drops to zero the object is instantly deallocated (Swift, n.d.). Android on the other hand makes use of Android Runtime (ART) which uses a Concurrent Copying (CC) garbage collection approach. The CC approach provides object allocation using a bump-pointer allocator. This approach performs heap defragmentation through concurrent object copying without introducing pauses to the app threads. On top of the CC, ART also supports Generational garbage collection for optimizing the collection of young objects (Android, n.d.-b).

The different approaches of garbage collection is a possible reason for the differences between the native and the cross-platform mobile application's RAM usage.

5.1.3 Research Question 3

The third research question aimed to understand why the cross-platform mobile application developed with Flutter exhibits higher CPU and RAM usage during the Lottie Animation execution compared to the native mobile applications:

> RQ3: Why does an application developed with the Flutter framework have a higher CPU and RAM usage when executing the Lottie Animation in comparison to the equivalent native application?

When looking at the mean CPU usage for the iOS mobile applications during the Lottie Animation, a significant performance difference can be observed. The iOS cross-platform mobile application exhibits a significantly higher mean CPU usage of 64%

compared to the iOS Native mobile application which only exhibited a mean CPU usage of 2.4%.

The lower mean CPU usage of the iOS Native mobile application is most likely due to the use of the official Lottie library. Due to the new rendering engine introduced in iOS Lottie 3.4.0 that uses Core Animation (GitHub, n.d.-e). The new engine renders graphics out-of-process with GPU hardware acceleration which means that once the animation begins, CPU resources are not consumed (GitHub, n.d.-e). Furthermore, The Lottie Animation in the native iOS is cached (GitHub, n.d.-d), which can be further supported by the pattern seen in the iOSNative_LottieA dataset graph in Appendix D, Figure D3. The peak CPU usage of the iOS Native mobile application can be observed in the beginning of the animation experiment, after which the subsequent data points exhibit mostly 0% CPU usage. This resulted in the low mean CPU usage of 2.4%, as can be seen in Appendix C, Figure C3.

In contrast to iOS, Flutter uses its own library for rendering Lottie animations (Dart Packages, n.d.). This means that Flutter has its own method for rendering Lottie animations which may not be as effective. This could potentially influence the CPU usage when executing the Lottie Animation in the cross-platform mobile application as it may not benefit from caching and the GPU hardware acceleration offered by the new Lottie engine in iOS.

An additional potential factor influencing the lower performance of the iOS Native mobile application could be an observed bug in the animation rendering. Although all animation elements used are officially supported on iOS devices according to LottieFiles (n.d.-c). The color tweening during the execution of the Lottie Animation did not occur. This bug could have affected the iOS Native mobile application's performance due to less rendering of changes.

However, it is worth noting that this bug is unlikely to have had such a profound impact on the performance that it could explain the significant difference observed between the native and cross-platform applications. Despite the color rendering issue in the native iOS app, the cross-platform application's performance was considerably worse, indicating that other factors are contributing to the performance difference.

The Android cross-platform application performed almost identical to the Android Native mobile application both in terms of the mean CPU usage as well as the peak CPU usage. This could be due to the Lottie library in the Android Cross-platform mobile application possibly having a closer integration with the Android device and the native API. This plausibility arises from the fact that both Android and Flutter are

developed by Google, implying a potentially closer interaction between the two that could cater to a deeper integration with the Android device as well as the native API.

The RAM usage in the cross-platform applications was consistently higher during the execution of the Lottie Animation, although it remained stable throughout all iterations. The garbage collector, mentioned in discussion to RQ2 as a potential factor for higher RAM usage, is unlikely to have had a significant effect on the RAM usage. This is because the Lottie Animation only involves one widget handling all the state changes, which would not be enough to put extensive load on the garbage collector.

The high RAM usage is more likely due to the use of the Lottie library. It is plausible that the implementation of Lottie Animation in the cross-platform mobile application may add extra computational work, thus increasing the memory allocation. The library itself may also require additional memory allocation, which would be reflected in the higher RAM usage recorded during the animation execution.

Moreover, the overall root of the higher RAM usage of the cross-platform mobile application could also be caused by the additional layer of abstraction between the code and the underlying hardware which allows that code to be compiled to multiple platforms. The high level Dart code needs to be translated to the low level native code which is likely to add to the RAM usage.

5.2 Method Discussion

This study aimed to compare the performance of cross-platform and native mobile application approaches using different animation implementations, and propose a solution for better performance. The choice of experimental study and the use of methods chosen was appropriate given the nature of the research questions. By incorporating a literature study, carefully designing experiments, and performing a data analysis, the purpose could be fulfilled and the research questions answered. Thereby contributing to the theoretical understanding in the field of mobile development and animation rendering.

Additionally, a more in depth study of the code architecture of the technologies would have provided a more detailed answer to the research questions but this was not achievable within the given time frame for this study. As a result, the proposed solution remains largely theoretical; emphasizing on the importance of strategic decisions in animation implementation and chosen technology rather than providing tangible coding solutions. The depth of these recommendations could be enhanced through a more extensive literature study that delves into more technical details.

The selection of popular platforms, frameworks and UI interactions is a strength of this study as it increases the generalizability of the findings. Furthermore, choosing the Flutter framework was a strategic decision to utilize the newly released version, Flutter 3, for this study. The absence of academic studies of this latest version offered a unique advantage for exploring its new improvements, particularly in the realm of animations (which also are minimally researched in terms of application performance). Of the improvements relevant for animation performance, Flutter 3 introduced partial repaint (re-rendering), improved opacity animations, and solved the jank issue (Flutter, n.d.-c; Flutter Team, 2022). While these improvements are more focused on user experience, it is still novel to explore what effects they had on the technical aspect of mobile application performance.

As mobile development is a fast paced industry, it was important to utilize the newest version and capabilities that were offered by Flutter so that the results of this study continue to be relevant. This is why the decision was made to use the new Impeller engine for rendering the UI for the iOS platform. This could be considered to be a flaw as the Android platform still used Skia, but this was a conscious decision made before the experiment that was based on Flutter Docs recommendations. During the time in which the experiment took place, Impeller was stable to use for iOS but not for Android (Flutter, n.d.-a).

The literature study effectively established the context and need for further investigation into the performance of cross-platform applications. The criteria set for the selection of relevant articles ensured the relevance of the articles used in the study. This approach was successful in providing valuable insights into state of art and the research gap. Although a more comprehensive literature study would greatly benefit the development of mobile applications, by finding strategies to optimize the code, as well as the ability to provide answers to the research questions by providing more technical details.

During the artifact development, one of the limitations encountered was the difficulty in keeping the Native Animation implementation consistent across different mobile applications, particularly in the native iOS mobile application. Although initially considered, the Core Animation framework was ruled out due to its level of integration not aligning with that of the Flutter framework. This decision subsequently narrowed the choices for implementing the Native Animation. The use of animation(_:) was initially seen as a promising solution but was later discovered to be deprecated in updated versions of iOS. Other alternatives could precisely replicate the straightforward tweening sequencing found in the Flutter framework. This led to a process of

meticulously ordering each tween in the code to correctly time the Native Animation. This methodology, while necessary, unfortunately led to reduced readability and maintainability of the code.

Another significant challenge in the native iOS mobile application was observed when attempting to loop the Native Animation. Around the 13th iteration, the animation exhibited janky behavior, causing the 15th iteration to skip entirely. To address this issue, an additional 0.5 seconds was added to each loop iteration, creating idle states that functioned as "rest" periods between iterations. This adjustment can clearly be seen in the iOSNative_NativeA graph provided in Appendix D, Figure D7. While the issue of jank was solved, thereby allowing the 15 iterations to complete, another difficulty arose. Due to the presence of idle states introduced, the method of extracting data points had to be adjusted specifically for the dataset. The extraction process of data points was modified to exclude the idle states, ensuring that only the relevant data points were considered for analysis.

The results presented by this study show a big performance difference between the mobile applications running on iOS versus Android. This could indicate a flaw in the method of profiling the performance. The profiling tools on iOS and Android IDE work similarly in theory but it is possible that they could differ in practice, especially by a human error. Other factors could be in play that the researcher, due to inexperience with profiling, are not familiar with. Thereby it is not possible to point at a specific issue that led to such differences.

6 Conclusions and Further Research

This chapter presents the conclusions from the study, the practical and scientific implications and provides suggestions for further research.

6.1 Conclusions

Reflecting on the problem statement and the research findings, this study contributes to understanding the performance gap of cross-platform mobile applications. An examination of CPU and RAM usage of animation implementations and development approaches has provided further evidence of the performance differences between native and cross-platform mobile applications.

The results reveal that although Flutter framework generally displayed increased CPU and RAM usage compared to native applications, this difference was less prominent when handling Lottie animations, particularly on Android devices. This implies that the selection of animation implementation approaches significantly impacts the performance of applications created using Flutter. The findings thereby suggest that a potential approach for mitigating the performance gap is through conscious selection of the widgets used within the Flutter framework or imported libraries.

6.1.1 Practical Implications

The findings from this study carry practical implications for mobile application developers. When deciding between native development and cross-platform development with Flutter, the trade-offs between the application performance and other factors (not covered in this study) such as development speed, scalability, user experience and cross-platform capability, must be considered. Moreover, the choice of animation to be implemented exerts a significant influence on the application's performance, as evidenced by the results obtained from the Native Animation Experiment and the Lottie Animation Experiment.

In instances where the performance is of importance, particularly in terms of CPU usage, choosing native development approach is preferred. However, if there is a greater need for development efficiency Flutter may be considered as the better alternative. Especially if a high performance framework or library is available for the specific use case, such as using Lottie for complex animations. However, the challenge might lie in finding such a framework or library which produces consistent performance for all generated cross-platform applications.

6.1.2 Scientific Implication

The findings from this study contribute to the growing body of literature on the performance gap of native and cross-platform mobile applications as well as animation rendering. In addition to confirming previous studies, the research extended the existing knowledge by comparing the performance of different animation implementations across both native and cross-platform mobile applications.

The results have also highlighted the role of animation implementation approach in determining the application's performance, thereby providing a new perspective for future research in the field.

6.2 Further Research

While this research has contributed to our understanding of mobile application performance under different development approaches and animation implementations, further research is needed to confirm and expand these findings.

Specifically, future studies should look to include other cross-platform development tools, such as React Native and Ionic, in the comparison. It would also be of interest to perform the experiment with an Android application developed with Jetpack Compose when the UI toolkit has reached a higher maturity. This would be interesting due to Jetpack Compose being more purely declarative in nature, than the combination of Kotlin and XML.

The performance study should be replicated when the new Impeller graphics engine has been released to Android devices by the Flutter team. Furthermore, the Impeller engine is claimed to solve visual UI issues observed in Skia Graphics Engine. This suggests that future replications of this study could also place an emphasis on evaluating user experience. Future studies may also extend the range of testing environments to include multiple devices with varying hardware specifications for a more comprehensive analysis.

Moreover, other types of animations and more complex animations could be investigated to further understand their impact on the performance of mobile applications.

7 References

Ahti, V., Hyrynsalmi, S., & Nevalainen, O.. (2016). An Evaluation Framework for Cross-Platform Mobile App Development Tools. https://doi.org/10.1145/2983468.2983484

Andersson, H. (2022). A Comparison of the Performance of an Android Application Developed in Native and Cross-Platform: Using the Native Android SDK and Flutter.

Andersson, D., & Axelsson, A. (2021). Evaluation of The Software Development Process for A Multi-Platform Solution in Flutter.

Android.(n.d.-a).*AAPT2*.Android Developer Documentation.Retrieved May 24, 2023, from https://developer.android.com/tools/aapt2

Android.(n.d.-b).*Debugging ART garbage collection*.Android Developer Documentation.Retrieved May 24, 2023, from https://source.android.com/docs/core/runtime/gc-debug

Android.(n.d.-c).*Canvas*.Android Developer Documentation.Retrieved May 24, 2023, from https://developer.android.com/reference/android/graphics/Canvas

Android.(n.d.-d).*Layouts in Views*.Android Developer Documentation.Retrieved May 24, 2023, from https://developer.android.com/develop/ui/views/layout/declaring-layout

Apple.(n.d.-a).*CAAnimation*.Apple Developer Documentation.Retrieved April 18, 2023, from https://developer.apple.com/documentation/quartzcore/caanimation

Apple.(n.d.-b).*CALayer*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/documentation/quartzcore/calayer

Apple.(n.d.-c).*Core Animation*.Apple Developer Documentation.Retrieved April 18, 2023, from https://developer.apple.com/documentation/quartzcore

Apple.(n.d.-d).*Core Data*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/documentation/coredata

Apple.(n.d.-e).*Foundation*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/documentation/foundation

Apple.(n.d.-f).*iOS*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/ios/

Apple.(n.d.-g).*SwiftUI*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/xcode/swiftui/

Apple.(n.d.-h).*UIkit*.Apple Developer Documentation.Retrieved April 18, 2023, from https://developer.apple.com/documentation/UIkit

Apple.(n.d.-i).*withAnimation(_:_:)*.Apple Developer Documentation.Retrieved April 18, 2023, from https://developer.apple.com/documentation/swiftui/withanimation(_:_:)

Apple.(n.d.-j).*Xcode*.Apple Developer Documentation.Retrieved May 24, 2023, from https://developer.apple.com/xcode/

Barros, L., Medeiros, F., Moraes, E., & Júnior, A. F. (2020). Analyzing the Performance of Apps Developed by using Cross-Platform and Native Technologies. In SEKE (pp. 186-191)

Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G.. (2019a). Animations in Cross-Platform Mobile Applications: An Evaluation of Tools, Metrics and Performance. Sensors, 19(9), 2081. https://doi.org/10.3390/s19092081

Biørn-Hansen A, Grønli TM, Ghinea G, Alouneh S (2019b) An empirical study of cross-platform mobile development in industry. Wirel Commun Mob Comput 2019

Biørn-Hansen, A., Rieger, C., Grønli, T.-M., Majchrzak, T. A., & Ghinea, G.. (2020). An empirical investigation of performance overhead in cross-platform mobile development frameworks. Empirical Software Engineering, 25(4), 2997–3040. https://doi.org/10.1007/s10664-020-09827-6

Boukhary, S., & Colmenares, E.. (2019). A Clean Approach to Flutter Development through the Flutter Clean Architecture Package. https://doi.org/10.1109/csci49370.2019.00211

Cheon, Y., & Chavez, C.. (2021). Converting Android Native Apps to Flutter Cross-Platform Apps. https://doi.org/10.1109/csci54926.2021.00355

Ciman, M., & Gaggi, O. (2017). An empirical analysis of energy consumption of cross-platform frameworks for mobile development. Pervasive and Mobile Computing, 39, 214-230.
Ciman, M., Gaggi, O., & Gonzo, N.. (2014). Cross-platform mobile development. https://doi.org/10.1145/2554850.2555104

Dart Packages.(n.d.).*lottie 2.3.2*.Retrieved May 30, 2023, from https://pub.dev/packages/lottie

Faiz, M. A., Kusumo, D. S., & Alibasa, M. J.. (2022). Flutter Framework Code Portability Measurement on Multiplatform Applications with ISO 9126. https://doi.org/10.1109/icoseit55604.2022.10030045

Flutter.(n.d.-a).*Impeller rendering engine*.Retrieved May 24, 2023, from https://docs.flutter.dev/perf/impeller

Flutter.(n.d.-b).*Introduction to widgets*.Retrieved May 24, 2023, from https://docs.flutter.dev/ui/widgets-intro

Flutter.(n.d.-c).*Start thinking declaratively.* Retrieved April 17, 2023, from https://docs.flutter.dev/development/data-and-backend/state-mgmt/declarative

Flutter Team.(2019, 4 Jan). *Flutter: Don't Fear the Garbage Collector*. Medium. https://medium.com/flutter/flutter-dont-fear-the-garbage-collector-d69b3ff1ca30

Flutter Team.(2022, 11 May). *What's new in Flutter 3*. Medium. https://medium.com/flutter/whats-new-in-flutter-3-8c74a5bc32d0

Flutter Team.(2023, 24 Jan). *What's new in Flutter 3.7*. Medium. https://medium.com/flutter/whats-new-in-flutter-3-7-38cbea71133c

GitHub.(n.d.-a).*Flutter*. Retrieved April 17, 2023, from https://github.com/flutter/flutter

GitHub.(n.d.-b).*Flutter/engine/impeller*. Retrieved may 24, 2023, from https://github.com/flutter/engine/blob/main/impeller/README.md

GitHub.(n.d.-c).*Ionic*. Retrieved April 17, 2023, from https://github.com/ionic-team/ionic-framework

GitHub.(n.d.-d).*lottie-ios/Sources/Public/AnimationCache /AnimationCacheProvider.swift*. Retrieved May 30, 2023, from https://github.com/airbnb/lottie-ios/blob/master/Sources/Public/AnimationCache/AnimationCacheProvider.swift

GitHub.(n.d.-e).*Lottie 3.4.0 now available, with significant performance improvements thanks to a new rendering engine powered by Core Animation*. Retrieved May 30, 2023, from https://github.com/airbnb/lottie-ios/discussions/1627

GitHub.(n.d.-f).*React Native*. Retrieved April 17, 2023, from https://github.com/facebook/react-native

Google. (2017, May 17). *Android announces support for Kotlin*. Google Developers Blog.https://android-developers.googleblog.com/2017/05/android-announces-support-for-kotlin.html

Google.(2021, July 28).*Jetpack Compose is now 1.0: announcing Android's modern toolkit for building native UI*.Google Developers Blog. https://android-developers.googleblog.com/2021/07/jetpack-compose-announcement.html

Huber, S., Demetz, L., & Felderer, M.. (2020). Analysing the Performance of Mobile Cross-platform Development Approaches Using UI Interaction Scenarios. In Communications in Computer and Information Science (pp. 40–57). Communications in Computer and Information Science. https://doi.org/10.1007/978-3-030-52991-8_3

Hudson, P.(2021, Nov 13).*Enabling high-performance Metal rendering with drawingGroup()*.Hacking with Swift. https://www.hackingwithswift.com/books/ios-swiftui/enabling-high-performance-metal-rendering-with-drawinggroup

LottieFiles(n.d.-a).*Create stunning animations with the LottieFiles plugin for Adobe After Effects.* Retrieved April 18, 2023, from https://lottiefiles.com/plugins/after-effects

LottieFiles.(n.d.-b).*Explore Lottie Animation Format, Lottie Players and the Latest Tools and Features*. Retrieved May 13, 2023, from https://lottiefiles.com/blog/about-lottie/explore-lottie-animation-format-lottie-players-latest-tools-and-features

LottieFiles.(n.d.-c).*Lottie Supported Features*. Retrieved May 25, 2023, from https://lottiefiles.com/supported-features

LottieFiles.(n.d.-d).*What is a Lottie?*. Retrieved April 17, 2023, from https://lottiefiles.com/what-is-lottie

Menegassi, A. A., & Endo, A. T.. (2020). Automated tests for cross-platform mobile apps in multiple configurations. IET Software, 14(1), 27–38. https://doi.org/10.1049/iet-sen.2018.5445

Nawrocki, P., Wrona, K., Marczak, M., & Sniezynski, B.. (2021). A Comparison of Native and Cross-Platform Frameworks for Mobile Applications. Computer, 54(3), 18–27. https://doi.org/10.1109/mc.2020.2983893

Scott, S.(2017, July 27).*See what's possible with After Effects*.Adobe.

https://helpx.adobe.com/after-effects/how-to/what-is-after-effects-cc.html?playlistPath=/services/playlist.helpx/products:SG_AFTEREFFECTS_1_1/learn-path:get-started/set-header:ccx-designer/playlist:orientation/en_us.json

Shah, K., Sinha, H., & Mishra, P.. (2019). Analysis of Cross-Platform Mobile App Development Tools. https://doi.org/10.1109/i2ct45611.2019.9033872

Sharma, S., Khare, S., Unival, V., & Verma, S.. (2022). Hybrid Development in Flutter and its Widgits. https://doi.org/10.1109/iccr56254.2022.9995973

Skia.(n.d.).*Documentation*.Skia Graphics Library.Retrieved May 24, 2023, from https://skia.org/docs/

Swift.(n.d.).*Automatic Reference Counting*.Swift Documentation.Retrieved May 24, 2023, from https://docs.swift.org/swift-book/documentation/the-swift-programming-language/automaticreferencecounting/

Vilcek, T., & Jakopec, T.. (2017). Comparative analysis of tools for development of native and hybrid mobile applications. https://doi.org/10.23919/mipro.2017.7973662

Willocx, M., Vossaert, J., & Naessens, V.. (2015). A Quantitative Assessment of Performance in Mobile App Development Tools. https://doi.org/10.1109/mobserv.2015.682

Xanthopoulos, S., & Xinogalos, S.. (2013). A comparative analysis of cross-platform development approaches for mobile applications. https://doi.org/10.1145/2490257.249029

8 Appendix

Appendix A The tweening sequence for each animation element.

Appendix B The results of the literature study conducted for the study.

Appendix C Lottie Animation and Native Animation experiment data.

Appendix D Lottie Animation and Native Animation experiment timeline graphs.

Appendix E The GitHub Repositories for the project.

Appendix A

Table A1: *Color property tweening sequence.*

Start value (Color)	**End value (Color)**	**Duration (seconds)**
blue	purple	0,8
purple	red	0,8
red	yellow	0,8
yellow	green	0,8
green	blue	0,8

Table A2: *Opacity property tweening sequence.*

Start value (double)	**End value (double)**	**Duration (seconds)**
0	0	0,96
0	1	0,2
1	0	0,2
0	0	1,68
0	1	0,2
1	1	0,8

Table A3: *Position (y-axis) property tweening sequence.*

Start value (double)	**End value (double)**	**Duration (seconds)**
0	400	1
400	400	0,2
400	300	0,8
300	300	0,2
300	400	0,8
400	400	1

Table A4: *Width (x-axis) property tweening sequence.*

Start value (double)	**End value (double)**	**Duration (seconds)**
50	40	0,4
40	30	0,4
30	80	0,2
80	60	0,4
60	40	0,6
40	60	0,2
60	45	0,8
45	70	0,2
70	50	0,4
50	50	0,4

Table A5: *Height (y-axis) property tweening sequence.*

Start value (double)	**End value (double)**	**Duration (seconds)**
50	60	0,4
60	70	0,4
70	20	0,2
20	40	0,4
40	50	0,6
50	35	0,2
35	60	0,8
60	30	0,2
30	50	0,4
50	50	0,4

Appendix B

Table B1: *The full list of the articles reviewed in the literature study conducted in this study.*

Authors	Year	Title	Purpose	Contribution	Mentioned in chapter
Andersson, H.	2022	A Comparison of the Performance of an Android Application Developed in Native and Cross-Platform: Using the Native Android SDK and Flutter.	To provide a confirmation of the performance differences between cross-platform applications and native applications	Provided a base for the problem statement and confirmed that the field of cross-platform application performance needs to be investigated further.	1 Introduction 1.1 Problem Statement 2.2 Artifact Development
Faiz, M. A., Kusumo, D. S., & Alibasa, M. J.	2022	Flutter Framework Code Portability Measurement on Multiplatform Applications with ISO 9126	To provide background information on Flutter	Provided understanding of the functionality of widgets in Flutter applications	3.3 Flutter
Sharma, S., Khare, S., Unival, V., & Verma, S.	2022	Hybrid Development in Flutter and its Widgits	To provide background information on Flutter	Provided information about the release date of the Dart programming language	3.3 Flutter
Andersson, D., &	2021	Evaluation of The Software Development Process for A	To provide background information on Flutter	Provided information about the Skia Graphics Engine used by	3.3 Flutter

Axelsson, A.		Multi-Platform Solution in Flutter		Flutter	
Cheon, Y., & Chavez, C.	2021	Converting Android Native Apps to Flutter Cross-Platform Apps	To provide detailed information about the layout components used in Android and Flutter	Provided technical insight in the elements used in Android and Flutter applications	2.2.1.1 Native Android App 3.1.1 Android 3.3 Flutter
Nawrocki, P., Wrona, K., Marczak, M., & Sniezynski, B.	2021	A Comparison of Native and Cross-Platform Frameworks for Mobile Applications	To provide further confirmation about the loss of performance in cross-platform applications as well as providing information about Google's Skia Engine	Confirmed that there exists a performance difference between native and cross-platform applications	1.1 Problem Statement 3.3 Flutter
Barros, L., Medeiros, F., Moraes, E., & Júnior, A. F.	2020	Analyzing the Performance of Apps Developed by using Cross-Platform and Native Technologies	To highlight the limitations and drawbacks of cross-platform frameworks	Provided insight into the trade-offs between code reusability and application performance	1 Introduction 3.1 Native development
Biørn-Hansen, A., Rieger, C.,	2020	An empirical investigation of performance overhead in cross-platform mobile	To provide background information about native development as well as	Provided insight in the current state of cross-platform maturity as well as background to native	1.1 Problem Statement

Grønli, T.-M., Majchrzak, T. A., & Ghinea, G.		development frameworks	different cross-platform development approaches	development and detailed insight into the different approaches available for cross-platform development	3.1 Native development 3.2 Cross-platform development 3.3 Flutter
Huber, S., Demetz, L., & Felderer, M.	2020	Analysing the Performance of Mobile Cross-platform Development Approaches Using UI Interaction Scenarios	To highlight the performance differences between native and cross-platform applications and to increase the reliability of the research gap through the findings from their study	Provided confirmation about the performance differences between native and cross-platform applications and acted as a base for defining the research gap through their findings of cross-platform application having a pattern of high CPU and RAM usage	1 Introduction 1.1 Problem Statement 3.1 Native development 3.2 Cross-platform development
Menegassi and Endo.	2020	Automated tests for cross-platform mobile apps in multiple configurations	To provided information about mobile device OS and the difficulties of testing cross-platform applications	Provided insight into mobile OS, the different hardware and configurations between mobile devices	1 Introduction 3.1 Native development 3.2 Cross-platform development
Biørn-Hansen, A., Grønli, T.-	2019 a	Animations in Cross-Platform Mobile Applications: An Evaluation of Tools, Metrics and Performance	To provide a deep understanding of the performance differences between native and cross-platform applications when	Provided insight into the performance differences between native and cross-platform applications and was the article where the research gap was	1 Introduction 1.1 Problem Statement 2.1 Designing the Animation

M., & Ghinea, G.			executing animations. Also provided a base for the method used for measuring and designing the animations.	initially found within the Future Work chapter of their study, where extending their research of application performance with animations with the use of the Flutter framework was recommended	2.2 Artifact Development 2.3.4 Metrics & Measuring Tools
Biørn-Hansen A, Grønli TM, Ghinea G, Alouneh S.	2019 b	An empirical study of cross-platform mobile development in industry.	To provide information about what the industry of mobile application regards as the issues with cross-platform development	Provided insight into the perceived issues with cross-platform development within the mobile application development industry	1.1 Problem Statement 3.2 Cross-platform development
Boukhary, S., & Colmenares, E.	2019	A Clean Approach to Flutter Development through the Flutter Clean Architecture Package	To provide background information on Flutter	Provided detailed information about the different types of compilation used by Flutter	3.3 Flutter
Shah, K., Sinha, H., & Mishra, P.	2019	Analysis of Cross-Platform Mobile App Development Tools	To provide information about the benefits and drawbacks of the shared code base of cross-platform frameworks	Provided information about the benefits and drawbacks with cross-platform applications	1 Introduction 3.2 Cross-platform development
Ciman, M., & Gaggi, O.	2017	An empirical analysis of energy consumption of	To provide information about the performance of cross-	Provided detailed insight in the different approaches available	1.1 Problem Statement

		cross-platform frameworks for mobile development.	platform applications and the different types of cross-platform development approaches	for cross-platform development	3.2 Cross-platform development
Vilcek, T., & Jakopec, T.	2017	Comparative analysis of tools for development of native and hybrid mobile applications	To provide background information on native application development	Provided explanation to the benefit of native application development	3.1 Native development
Ahti, V., Hyrynsalmi, S., & Nevalainen, O.	2016	An Evaluation Framework for Cross-Platform Mobile App Development Tools	To explain the effects of high RAM usage	Provided insight into how the usability of mobile devices are affected by the RAM usage	1 Introduction
Willocx, M., Vossaert, J., & Naessens, V.	2015	A Quantitative Assessment of Performance in Mobile App Development Tools	To explain the cost accompanied by not being able to share code between different platforms and what the effects of high CPU usage has on mobile devices	Provided insight into how the usability of mobile devices are affected by the CPU usage	1 Introduction 1.1 Problem Statement
Ciman, M., Gaggi, O., & Gonzo, N.	2014	Cross-platform mobile development	To introduce the use and benefits of cross-platform frameworks	Provided insight in to the benefits of cross-platform frameworks	1 Introduction

Xanthopoulos & Xinogalos.	2013	A comparative analysis of cross-platform development approaches for mobile applications	To provide a clear definition of goals of cross-platform development	Provided context about the aim of cross-platform frameworks	1 Introduction

Table B2: *The official websites used for gathering information and understanding about the technologies and tools used in the study.*

Source	Year	Topic	URL	Contribution	Mentioned in chapter
Android Developer Documentation	n.d	AAPT2	https://developer.android.com/tools/aapt2	Provided insight in how the Android SDK parses XML-files into view objects	5.1.1 Research Question 1
Android Developer Documentation	n.d.	Debugging ART garbage collection	https://source.android.com/docs/core/runtime/gc-debug	Provided insight into the Android garbage collection	5.1.2 Research Question 2

Android Developer Documentation	n.d.	Canvas	https://developer.android.com/reference/android/graphics/Canvas	Provided insight into how elements are drawn in the views of Android applications	5.1.2 Research Question 2
Android Developer Documentation	n.d	Layouts in Views	https://developer.android.com/develop/ui/views/layout/declaring-layout	Provided insight into how layouts work in Android	5.1.1 Research Question 1
Apple Developer Documentation	n.d	CAAnimation	https://developer.apple.com/documentation/quartzcore/caanimation	Provided insight to alternative solutions for implementing animation	2.2.1.2 Native iOS App
Apple Developer Documentation	n.d.	CALayer	https://developer.apple.com/documentation/quartzcore/calayer	Provided insight into how iOS draws element in the views	5.1.2 Research Question 2
Apple Developer Documentation	n.d	Core Animation	https://developer.apple.com/documentation/quartzcore	Provided insight to alternative solutions for implementing animation	2.2.1.2 Native iOS App

Apple Developer Documentation	n.d	Core Data	https://developer.apple.com/documentation/coredata	Provided insight into the core frameworks used for iOS app development	3.1.2 iOS
Apple Developer Documentation	n.d	Foundation	https://developer.apple.com/documentation/foundation	Provided insight into the core frameworks used for iOS app development	3.1.2 iOS
Apple Developer Documentation	n.d	iOS	https://developer.apple.com/ios/	Provided insight into the iOS SDK	3.1.2 iOS
Apple Developer Documentation	n.d	SwiftUI	https://developer.apple.com/xcode/swiftui/	Provided detailed information about the SwiftUI framework	5.1.2 Research Question 1
Apple Developer Documentation	n.d	UIkit	https://developer.apple.com/documentation/UIkit	Provided insight into the core frameworks used for iOS app development	3.1.2 iOS
Apple Developer Documentation	n.d	withAnimation(_:_:)	https://developer.apple.com/documentation/swiftui/withanimation(_:_:)	Provided insight into the function used for creating the iOS animation	2.2.1.2 Native iOS App

Apple Developer Documentation	n.d	Xcode	https://developer.apple.com/xcode/	Provided insight into the native IDE used for iOS development	3.1.2 iOS
Dart Packages	n.d	lottie 2.3.2	https://pub.dev/packages/lottie	Provided insight into how FLutter renders Lottie animations	5.1.3 Research Question 3
Flutter	n.d	Impeller rendering engine	https://docs.flutter.dev/perf/impeller	Provided insight into the graphics engine used by Flutter on iOS devices	5.1.2 Research Question 2 5.2 Method Discussion
Flutter	n.d	Introduction to widgets	https://docs.flutter.dev/ui/widgets-intro	Provided an detailed insight into Flutters widgets	5.1.2 Research Question 2
Flutter	n.d	Start thinking declaratively	https://docs.flutter.dev/development/data-and-backend/state-mgmt/declarative	Provided information about the Flutter framework	3.3 Flutter
Flutter Team	2019	Flutter: Don't Fear the Garbage Collector	https://medium.com/flutter/flutter-dont-fear-the-garbage-collector-d69b3ff1ca30	Provided information about the Flutter frameworks use of garbage collection	5.1.2 Research Question 2
Flutter Team	2022	What's new in Flutter 3	https://medium.com/flutter/whats-new-in-flutter-3-	Provided information about the Flutter 3 release and and what's been updated	1 Introduction

			8c74a5bc32d0		
Flutter Team	2023	What's new in Flutter 3.7	https://medium.com/flutter/whats-new-in-flutter-3-7-38cbea71133c	Provided insight into the new graphics rendering engine used by Flutter	5.1.2 Research Question 2
GitHub	n.d	Flutter	https://github.com/flutter/flutter	Provided information about the popularity of the Flutter repository in GitHub	1 Introduction
GitHub	n.d	Flutter/engine/impeller	https://github.com/flutter/engine/blob/main/impeller/README.md	Provided insight into the new graphics rendering engine used by Flutter	5.1.2 Research Question 2
GitHub	n.d	Ionic	https://github.com/ionic-team/ionic-framework	Provided information about the popularity of the Ionic repository in GitHub	1 Introduction
GitHub	n.d	lottie-ios/Sources/Public/AnimationCache/AnimationCacheProvider.swift	https://github.com/airbnb/lottie-ios/blob/master/Sources/Public/AnimationCache/AnimationCacheProvider.swift	Provided confirmation about the reason of the caching of the Lottie Animation in native iOS	5.1.3 Research Question 3
GitHub	n.d	Lottie 3.4.0 now available, with significant	https://github.com/airbnb/lottie-ios/discussions/1627	Provided information about the Lottie library for iOS	5.1.3 Research Question 3

		performance improvements thanks to a new rendering engine powered by Core Animation			
GitHub	n.d	React Native	https://github.com/facebook/react-native	Provided information about the popularity of the React Native repository in GitHub	1 Introduction
Google Developers Blog	2017	Android announces support for Kotlin	https://android-developers.googleblog.com/2017/05/android-announces-support-for-kotlin.html	Provided information about the official language of Android development	2.2.1.1 Native Android App 3.1.1 Android
Google	2021	Jetpack Compose is now 1.0: announcing Android's modern toolkit for building native UI	https://android-developers.googleblog.com/2021/07/jetpack-compose-announcement.html	Provided information about the UI toolkit Jetpack Compose	2.2.1.1 Native Android App
Hudson	2021	Enabling high-performance Metal rendering with drawingGroup()	https://www.hackingwithswift.com/books/ios-swiftui/enabling-high-performance-metal-rendering-with-drawinggroup	Provided insight into the usage of Core Animation	3.1.2 iOS

LottieFiles	n.d	Create stunning animations with the LottieFiles plugin for Adobe After Effects	https://lottiefiles.com/plugins/after-effects	Provided information about the official way of creating Lottie animations	2.2.1 Lottie Animation Implementation
LottieFiles	n.d	Explore Lottie Animation Format, Lottie Players and the Latest Tools and Features	https://lottiefiles.com/blog/about-lottie/explore-lottie-animation-format-lottie-players-latest-tools-and-features	Provided information about Lottie Animations and their usability	1.1 Problem Statement
LottieFiles	n.d	Lottie Supported Features	https://lottiefiles.com/supported-features	Provided information about which animation elements that each platform supported	5,1,3 Research Question 3
LottieFiles	n.d	What is a Lottie?	https://lottiefiles.com/what-is-lottie	Provided detailed information about Lottie Animations and their properties	3.4 Lottie Animation
Skia	n.d.	Documentation	https://skia.org/docs/	Provided insight into the Skia graphics engine	3.1.1 Android
Swift	n.d.	Automatic Reference Counting	https://docs.swift.org/swift-book/documentation/the-swift-programming-language/automaticreferencecounting/	Provided insight into the garbage collection of iOS	5.1.2 Research Question 2

Device	Data Point	CPU %	RAM (MB)
Android	1	5	72,4
	2	6	72,9
Approach	3	9	72,5
Native	4	9	71,9
	5	6	72,6
Dataset	6	9	72,7
AndroidNative_LottieA	7	7	72,3
	8	9	72,3
	9	6	72,4
	10	5	72,4
	11	6	72,6
	12	5	72,5
	13	6	72,5
	14	5	72,7
	15	6	72,7
	16	5	72,8
	17	5	72,9
	18	5	73,1
	19	5	73,1
	20	5	73,2
	21	6	73,2
	22	4	73,5
	23	6	73,5
	24	7	73,6
	25	5	73,7
	26	6	73,8
	27	5	73,9
	28	6	74
	29	5	74,1
	30	5	74,1
	31	6	74,2
	32	5	74,3
	33	5	74,4
	34	6	74,5
	35	5	74,5
	36	5	74,6
	37	6	74,7
	38	6	74,8
	39	6	74,9
	40	5	75
	41	4	75
	42	5	75,1
	43	4	75,2
	44	5	75,3
	45	4	75,4
	46	6	75,5
	47	5	75,6
	48	6	75,7
	49	5	75,7
	50	5	75,8
	51	6	75,9
	52	5	76
	53	5	76,1
	54	5	76,2
	55	6	76,3
	56	6	76,3
	57	5	76,3
	58	5	76,4
	59	7	76,4
	60	5	76,5

CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
9	5,633333333	76,5	74,20833333

Pre CPU %	Post CPU %	Pre RAM	Post RAM
0	0	70.7	76.5

Figure C1: *Data contained in dataset AndroidNative_LottieA*

Device	Data Point	CPU %	RAM (MB)		CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
Android	1	4	150,9		9	5,483333333	154,9	151,0816667
	2	5	151,2					
Approach	3	5	152,9		Pre CPU %	Post CPU %	Pre RAM	Post RAM
Cross-platform	4	7	153,4		0	0	150,4	149,6
	5	6	154,4					
Dataset	6	7	153,9					
AndroidFlutter_LottieA	7	7	154,6					
	8	5	154,1					
	9	6	154,7					
	10	5	154,3					
	11	6	154,9					
	12	5	154,4					
	13	6	154,9					
	14	5	154,4					
	15	5	150,7					
	16	6	150,2					
	17	5	150,7					
	18	5	150,3					
	19	5	150,8					
	20	7	149,8					
	21	6	150,6					
	22	5	149,9					
	23	5	150,7					
	24	5	149,9					
	25	5	150,7					
	26	7	150					
	27	6	150,8					
	28	5	150					
	29	5	150,8					
	30	5	150,1					
	31	6	150,9					
	32	5	150,1					
	33	5	150,9					
	34	7	150,2					
	35	5	150,1					
	36	5	150,2					
	37	6	150,1					
	38	5	150,3					
	39	5	149,8					
	40	5	150,4					
	41	7	149,9					
	42	6	150,5					
	43	5	150					
	44	5	150,5					
	45	5	150					
	46	6	150,5					
	47	5	149,7					
	48	5	150,2					
	49	7	149,8					
	50	5	150,3					
	51	5	149,9					
	52	5	150,3					
	53	5	149,9					
	54	5	150,5					
	55	5	150					
	56	7	150,4					
	57	5	149,7					
	58	6	150,5					
	59	5	149,7					
	60	5	150,6					

Figure C2: *Data contained in dataset AndroidFlutter_LottieA*

Device	Data Point	CPU %	RAM (MB)
IOS	1	82,5	5,69
	2	0	5,67
Approach	3	0	5,66
Native	4	0	5,66
	5	0	5,65
Dataset	6	0	5,65
iOSNative_LottieA	7	10	5,61
	8	10	5,6
	9	0	5,6
	10	0	5,6
	11	0	5,6
	12	0	5,6
	13	0	5,6
	14	0	5,6
	15	0	5,6
	16	0	5,6
	17	0	5,6
	18	0	5,6
	19	0	5,6
	20	0	5,6
	21	0	5,6
	22	0	5,6
	23	0	5,6
	24	0	5,6
	25	0	5,6
	26	0	5,6
	27	0	5,6
	28	0	5,6
	29	0	5,6
	30	0	5,6
	31	0	5,6
	32	0	5,6
	33	0	5,6
	34	0	5,6
	35	0	5,6
	36	0	5,6
	37	0	5,6
	38	0	5,6
	39	0	5,6
	40	0	5,6
	41	0	5,6
	42	0	5,6
	43	0	5,6
	44	0	5,6
	45	0	5,6
	46	0	5,6
	47	0	5,6
	48	0	5,6
	49	0	5,6
	50	0	5,6
	51	0	5,6
	52	0	5,6
	53	0	5,59
	54	20	5,59
	55	0	5,59
	56	0	5,59
	57	0	5,59
	58	10	5,59
	59	0	5,59
	60	10	5,59

CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
82,5	2,375	5,69	5,605166667
Pre CPU %	Post CPU %	Pre RAM	Post RAM
0	0	5,7	5,59

Figure C3: *Data contained in dataset iOSNative_LottieA*

Device	Data Point	CPU %	RAM (MB)		CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
IOS	1	73,8	28,98		100	64,00666667	30,09	29,9845
	2	74,3	29,02					
Approach	3	60	30,05		Pre CPU %	Post CPU %	Pre RAM	Post RAM
Cross-platform	4	67,1	30,05		0	0	27,91	29,98
	5	68,3	30,06					
Dataset	6	65	30,06					
iOSFlutter_LottieA	7	71,2	30,06					
	8	66,2	30,06					
	9	72,9	30,08					
	10	61,3	30,07					
	11	65	30,06					
	12	70	30,08					
	13	60	30,02					
	14	63,8	30,02					
	15	64,3	30,02					
	16	62,5	30,02					
	17	66,2	30,02					
	18	61,3	30,02					
	19	70	30,02					
	20	61,3	30,01					
	21	56,2	30					
	22	68,6	29,99					
	23	60	29,99					
	24	61,3	30					
	25	65,7	30,01					
	26	61,3	30,01					
	27	68,8	30,01					
	28	61,3	30,01					
	29	67,1	29,99					
	30	67,5	30					
	31	65	30					
	32	68,6	29,98					
	33	56,2	29,98					
	34	62,5	30,01					
	35	58,8	30,01					
	36	61,3	30					
	37	62,5	30					
	38	61,3	29,99					
	39	61,3	30					
	40	58,6	30					
	41	61,4	30,01					
	42	60	30,01					
	43	68,8	30,01					
	44	57,1	30,03					
	45	67,5	30,02					
	46	65,7	29,99					
	47	70	29,99					
	48	61,3	30,02					
	49	52,5	30,03					
	50	63,8	30,03					
	51	70	30,03					
	52	61,3	30,03					
	53	62,5	30,03					
	54	60	30,03					
	55	62,5	29,99					
	56	61,3	29,99					
	57	61,4	30,03					
	58	72,5	30,03					
	59	56,2	30,03					
	60	66,2	29,98					

Figure C4: *Data contained in dataset iOSFlutter_LottieA*

Device	Data Point	CPU %	RAM (MB)
Android	1	4	71,7
	2	5	71
Approach	3	4	70,4
Native	4	3	70,5
	5	5	70,6
Dataset	6	4	70,7
AndroidNative_NativeA	7	3	70,8
	8	5	71
	9	3	71
	10	5	71,1
	11	4	71,2
	12	3	71,3
	13	5	71,4
	14	4	71,5
	15	4	71,6
	16	5	71,7
	17	5	71,8
	18	3	71,9
	19	4	72
	20	4	72,1
	21	5	72,2
	22	4	72,3
	23	5	72,4
	24	5	72,5
	25	5	72,6
	26	3	72,7
	27	5	72,8
	28	3	72,9
	29	4	73
	30	4	73,1
	31	4	73,2
	32	4	73,3
	33	3	73,4
	34	3	73,5
	35	4	73,6
	36	4	73,7
	37	3	73,8
	38	4	73,9
	39	4	74
	40	4	74,1
	41	3	74,2
	42	3	74,3
	43	3	74,4
	44	4	74,5
	45	4	74,6
	46	4	74,7
	47	4	74,8
	48	3	74,9
	49	3	75
	50	4	75,1
	51	5	75,2
	52	3	75,3
	53	4	75,4
	54	3	75,5
	55	4	75,6
	56	3	71,9
	57	5	70,9
	58	3	71
	59	4	71,1
	60	4	71

CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
5	3,916666667	75,6	72,795

Pre CPU %	Post CPU %	Pre RAM	Post RAM
0	0	59	71

Figure C5: *Data contained in dataset AndroidNative_NativeA*

Device	Data Point	CPU %	RAM (MB)		CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
Android	1	4	142,2		8	6,05	146,2	141,7933333
	2	5	143,1					
Approach	3	7	143,8		Pre CPU %	Post CPU %	Pre RAM	Post RAM
Cross-platform	4	5	144,4		0	0	140,9	141,6
	5	8	145,7					
Dataset	6	5	145,5					
AndroidFlutter_NativeA	7	6	145,3					
	8	7	145,4					
	9	7	145,2					
	10	7	145					
	11	6	146,1					
	12	7	142,3					
	13	7	141,4					
	14	6	141,2					
	15	7	141,3					
	16	7	141					
	17	6	140,7					
	18	6	141,8					
	19	5	141,5					
	20	7	141,3					
	21	7	141					
	22	6	141,1					
	23	6	140,8					
	24	7	140,7					
	25	5	141,7					
	26	6	141,4					
	27	5	141,2					
	28	7	141,4					
	29	7	141,1					
	30	5	140,8					
	31	5	141,8					
	32	5	141,6					
	33	6	141,4					
	34	6	141,1					
	35	7	141,3					
	36	7	140,9					
	37	7	140,7					
	38	7	141,8					
	39	6	141,5					
	40	5	141,3					
	41	6	141					
	42	6	141,2					
	43	5	140,9					
	44	6	140,6					
	45	7	141,2					
	46	5	141					
	47	6	140,8					
	48	5	141					
	49	6	140,7					
	50	7	140,5					
	51	5	141,5					
	52	6	141,2					
	53	6	141					
	54	5	140,8					
	55	6	140,9					
	56	7	140,7					
	57	6	140,4					
	58	5	141,4					
	59	6	141,1					
	60	5	140,9					

Figure C6: *Data contained in dataset AndroidFlutter_NativeA*

Device	Data Point	CPU %	RAM (MB)		CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
IOS	1	90	5,7		100	30,335	6,19	6,077
	2	14,3	5,7					
Approach	3	12,9	5,68		Pre CPU %	Post CPU %	Pre RAM	Post RAM
Native	4	16,7	5,67		0	0	5,57	6,06
	5	100	5,68					
Dataset	6	15	5,68					
iOSNative_NativeA	7	11,2	5,64					
	8	23,3	5,64					
	9	60	5,53					
	10	12,9	6,19					
	11	16,2	6,19					
	12	20	6,19					
	13	96,7	6,07					
	14	17,1	6,18					
	15	15	6,18					
	16	3,3	6,18					
	17	40	6,18					
	18	14,3	6,18					
	19	11,4	6,18					
	20	16,7	6,17					
	21	85	6,06					
	22	13,3	6,18					
	23	11,7	6,18					
	24	5	6,17					
	25	70	6,18					
	26	15,7	6,18					
	27	14,3	6,18					
	28	11,4	6,17					
	29	96	6,18					
	30	11,2	6,18					
	31	12,9	6,17					
	32	8,6	6,17					
	33	78,3	6,06					
	34	11,4	6,18					
	35	10	6,17					
	36	3,3	6,17					
	37	83,3	6,07					
	38	14,3	6,17					
	39	12,9	6,17					
	40	10	6,17					
	41	80	6,07					
	42	15	6,17					
	43	13,3	6,17					
	44	15	6,17					
	45	83,3	6,05					
	46	15	6,18					
	47	14,3	6,18					
	48	14	6,18					
	49	88	6,07					
	50	13,3	6,17					
	51	11,7	6,17					
	52	13,3	6,06					
	53	96	6,06					
	54	8,8	6,17					
	55	11,4	6,17					
	56	10	6,04					
	57	81,7	6,06					
	58	18,8	6,17					
	59	18,3	6,17					
	60	13,3	6,17					

Figure C7: *Data contained in dataset iOSNative_NativeA*

Device	Data Point	CPU %	RAM (MB)		CPU % (Peak)	CPU % (Mean)	RAM (Peak)	RAM (Mean)
IOS	1	65	28,71		77,1	55,15666667	29,81	29,66683333
	2	55	29,75					
Approach	3	64,3	29,78		Pre CPU %	Post CPU %	Pre RAM	Post RAM
Cross-platform	4	50	29,75		0	0	28,6	29,7
	5	67,5	29,81					
Dataset	6	57,1	29,78					
iOSFlutter_NativeA	7	55	29,75					
	8	60	29,8					
	9	57,1	29,76					
	10	53,8	29,81					
	11	63,3	29,77					
	12	53,8	29,7					
	13	56,2	29,67					
	14	47,1	29,62					
	15	53,8	29,67					
	16	53,8	29,64					
	17	58,8	29,68					
	18	52,9	29,63					
	19	43,8	29,68					
	20	51,2	29,63					
	21	61,7	29,69					
	22	60	29,62					
	23	52,5	29,68					
	24	55,7	29,62					
	25	67,5	29,67					
	26	51,4	29,64					
	27	58,8	29,69					
	28	52,9	29,64					
	29	58,6	29,69					
	30	56,2	29,65					
	31	58,3	29,71					
	32	55	29,66					
	33	53,8	29,71					
	34	48,8	29,66					
	35	50	29,62					
	36	45	29,68					
	37	55	29,64					
	38	58,8	29,69					
	39	54,3	29,65					
	40	56,7	29,7					
	41	55	29,67					
	42	48,6	29,63					
	43	52,9	29,68					
	44	52,5	29,64					
	45	58,8	29,69					
	46	57,1	29,65					
	47	64,3	29,65					
	48	55	29,67					
	49	56,2	29,64					
	50	56,7	29,69					
	51	51,2	29,65					
	52	52,9	29,7					
	53	54,3	29,66					
	54	52,5	29,63					
	55	53,8	29,68					
	56	51,4	29,65					
	57	60	29,71					
	58	52,5	29,65					
	59	47,5	29,7					
	60	45,7	29,67					

Figure C8: *Data contained in dataset AndroidNative_LottieA*

Appendix D

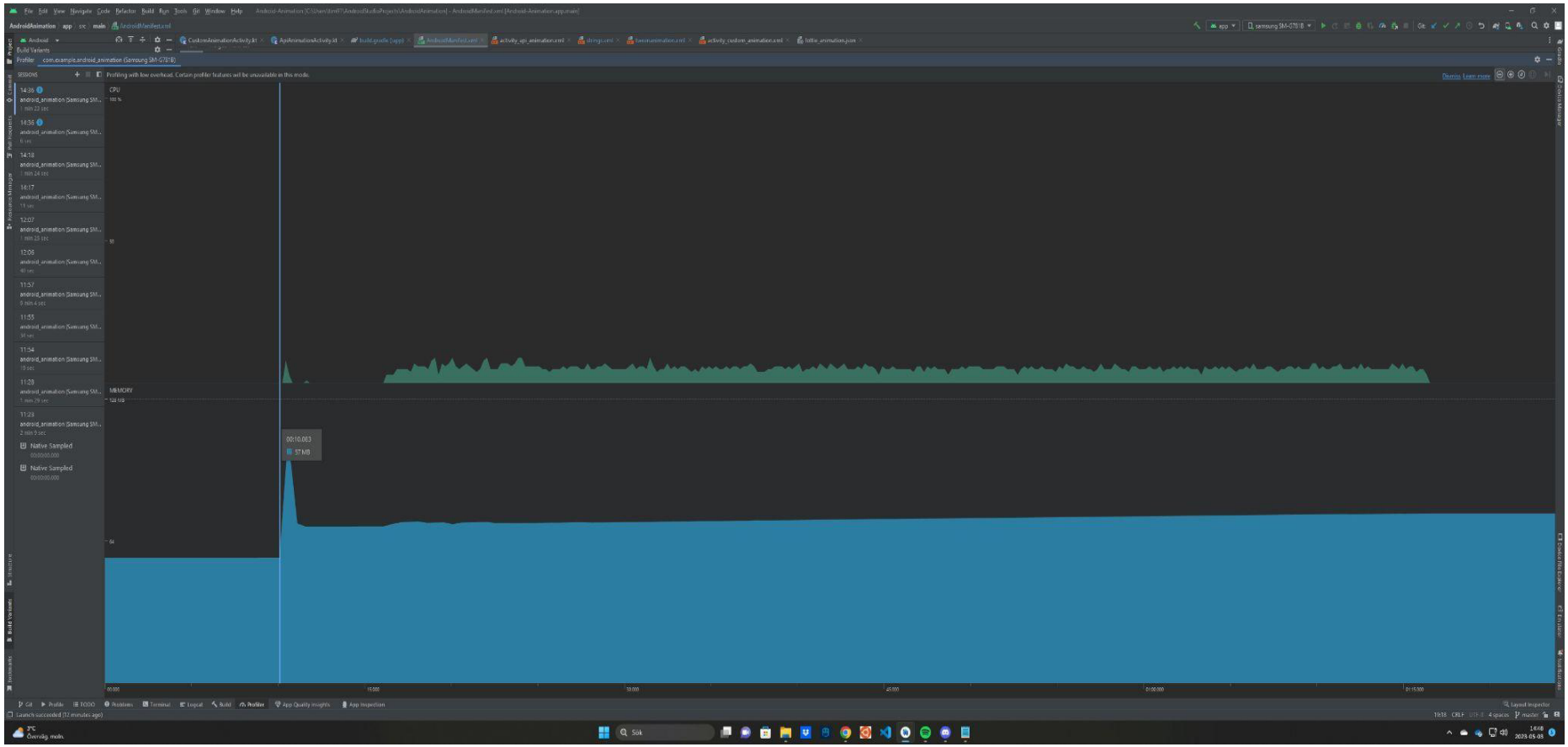

Figure D1: *Graph generated for dataset AndroidNative_LottieA*

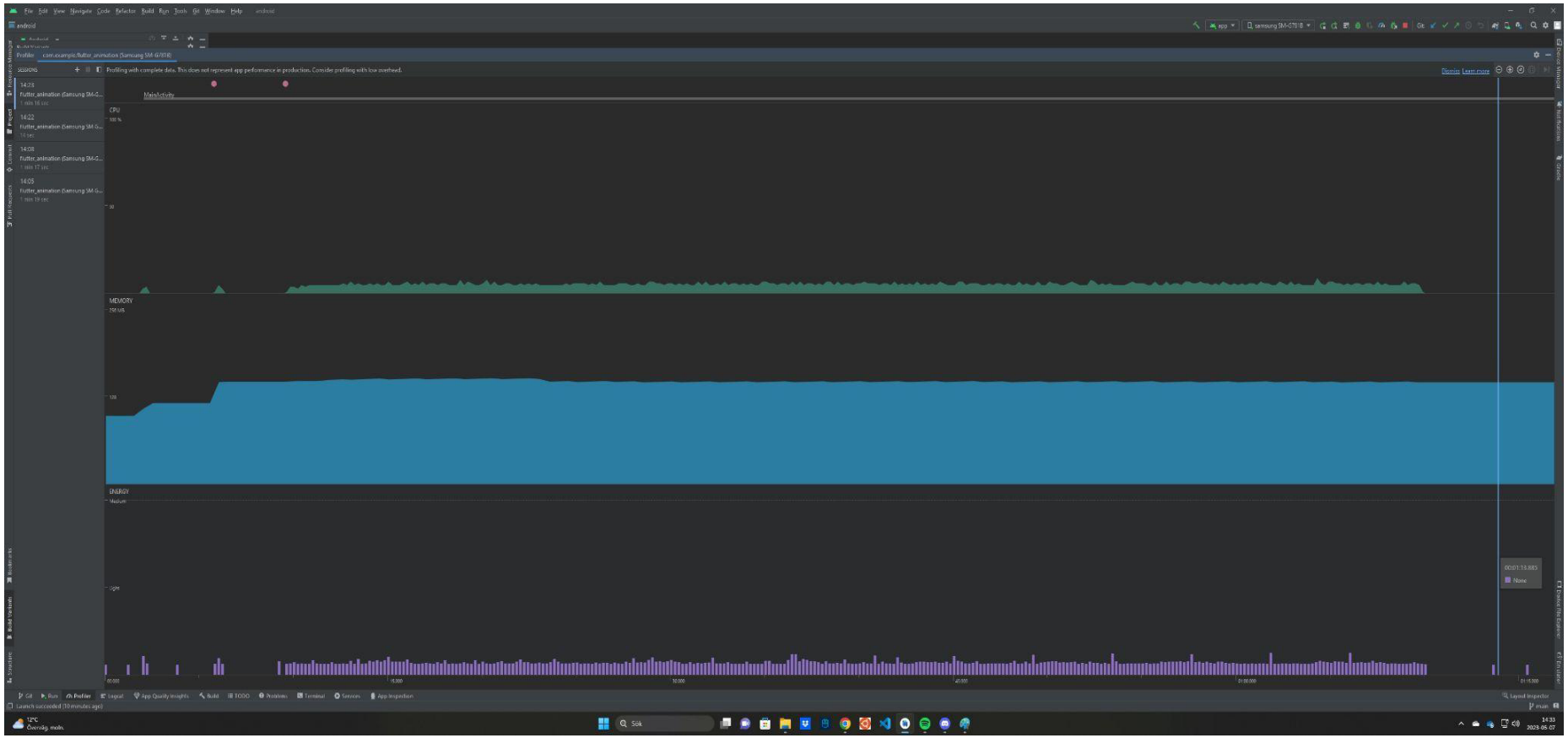

Figure D2: *Graph generated for dataset AndroidFlutter_LottieA*

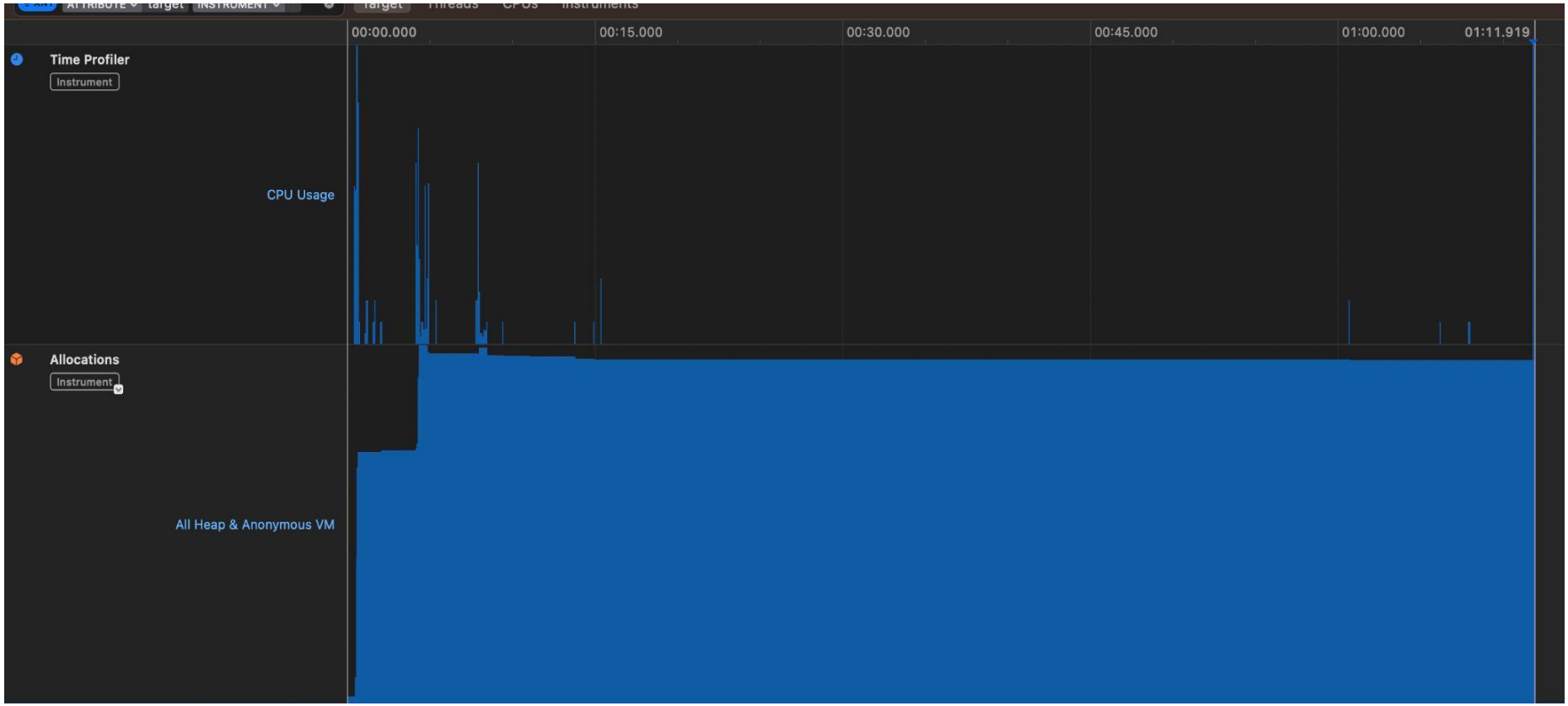

Figure D3: *Graph generated for dataset iOSNative_LottieA*

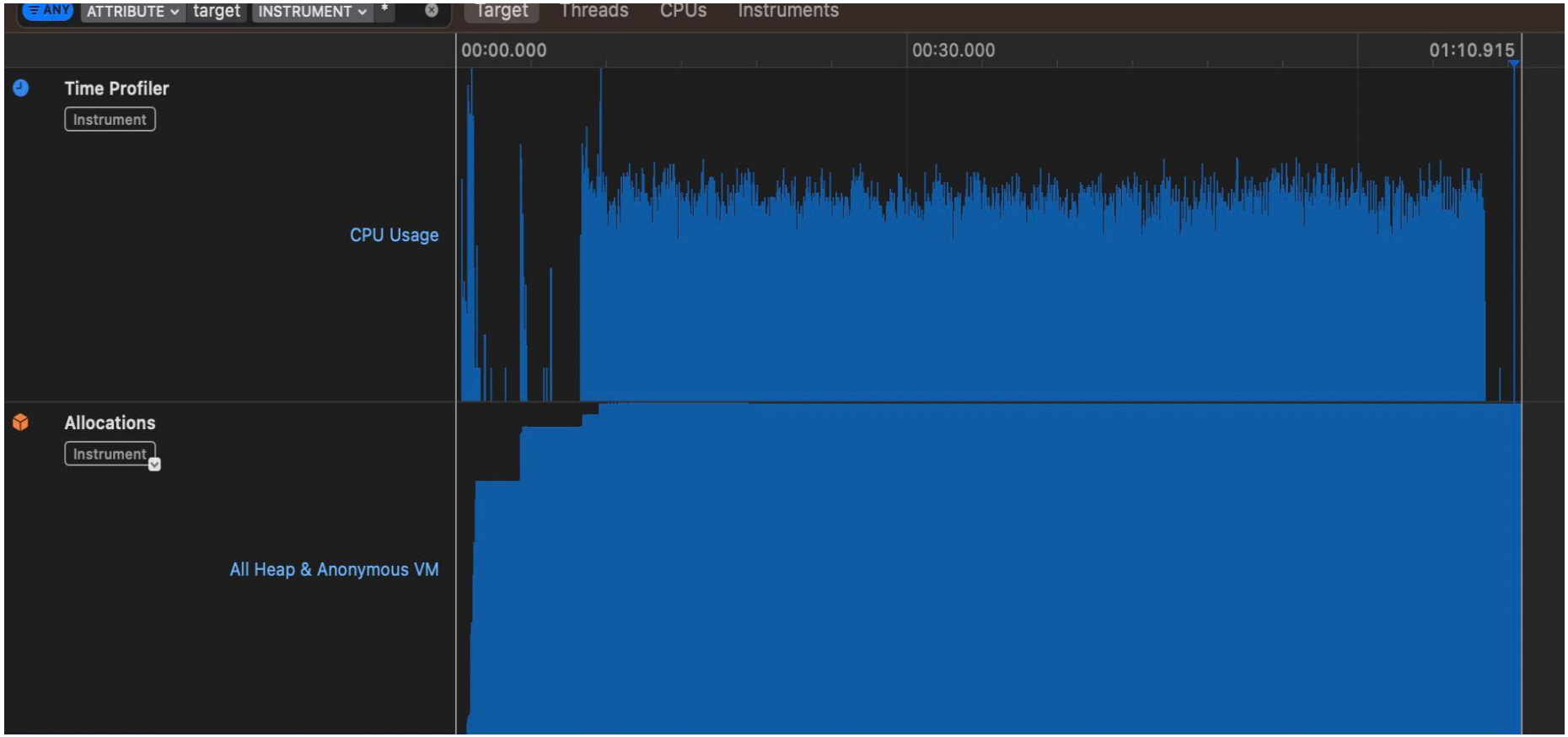

Figure D4: *Graph generated for dataset iOSFlutter_LottieA*

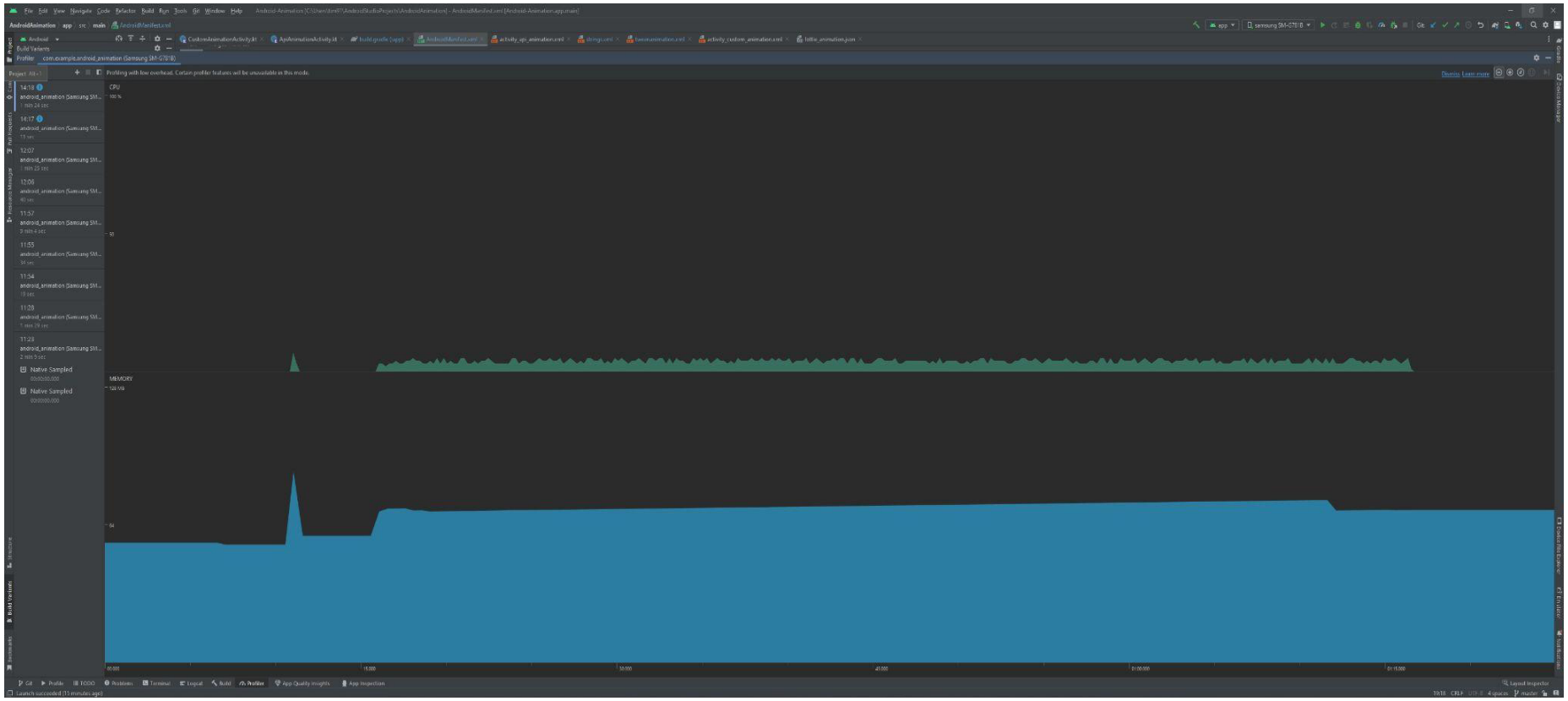

Figure D5: *Graph generated for dataset AndroidNative_NativeA*

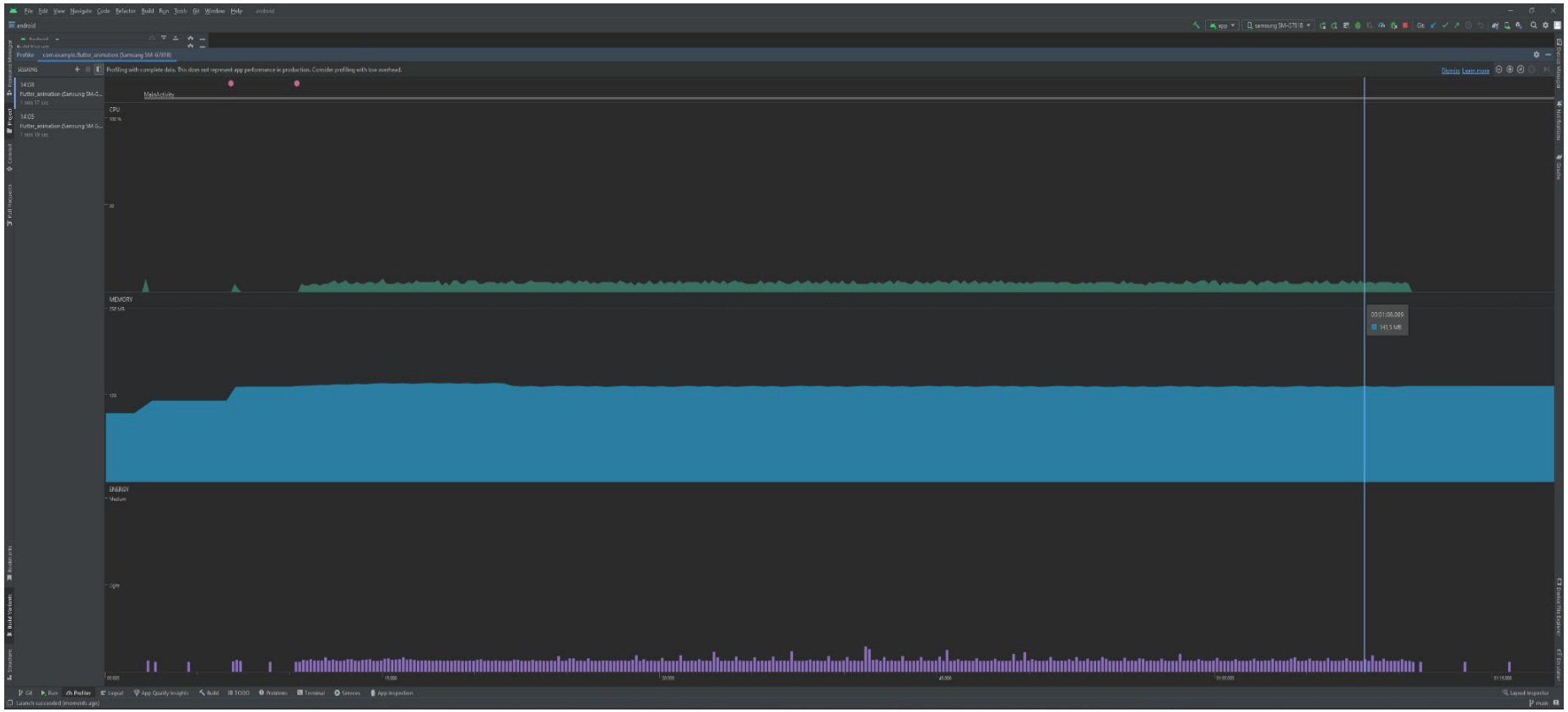

Figure D6: *Graph generated for dataset AndroidFlutter_NativeA*

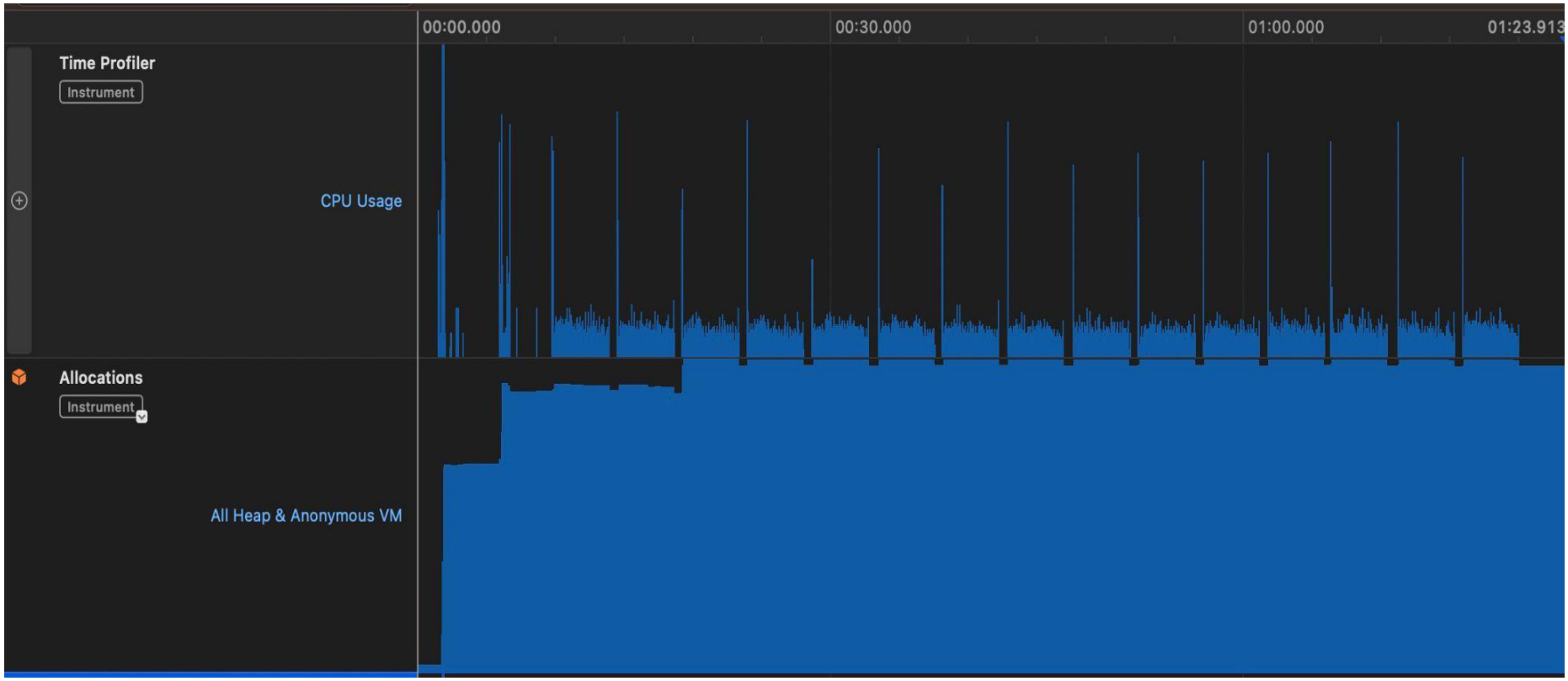

Figure D7: *Graph generated for dataset iOSNative_NativeA*

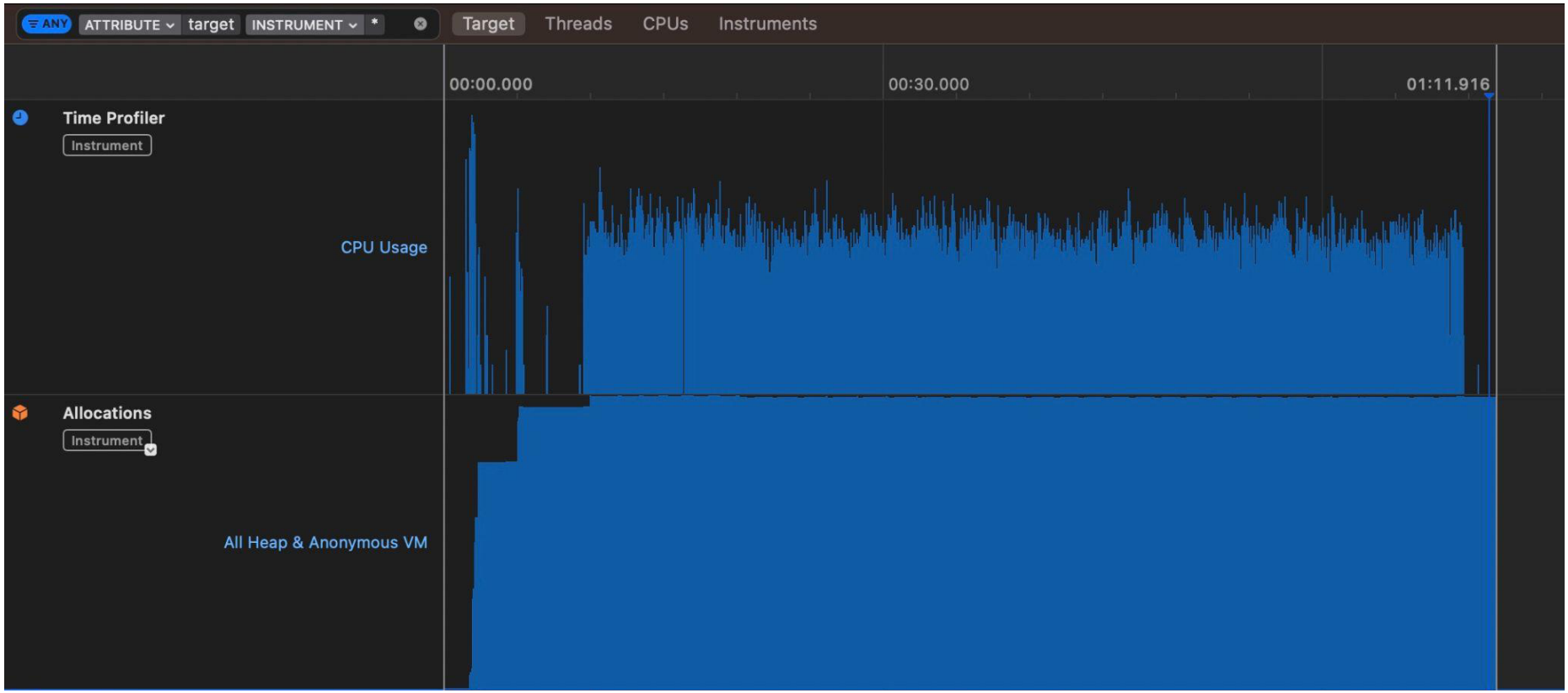

Figure D8: *Graph generated for dataset iOSFlutter_NativeA*

Appendix E

GitHub Repositories

Native Android application
https://github.com/Magoulo/Android-Animation

Native IOS application
https://github.com/Magoulo/iOS-Animation

Flutter application
https://github.com/Magoulo/Flutter-Animation